Visual Layer for Camtek

User Guide and Documentation

Contents

About This Documentation

This documentation is specifically designed for the Camtek Visual Layer implementation. Camtek uses Visual Layer for manufacturing quality control, where the system processes output to rapidly train defect detection models. This implementation is deployed on-premises, allowing direct access to machine output folders rather than traditional cloud uploads.

SECTION I

An Overview of Visual Layer

Integrating Visual Layer in Manufacturing Workflows

Visual Layer is a unified platform, designed to help teams manage, curate, and analyze visual data at scale, streamlining the entire data lifecycle. The platform provides centralized tools for exploring datasets, identifying patterns, maintaining data quality, and collaborating across teams.

> **New to Visual Layer terminology?** A comprehensive glossary with detailed definitions, examples, and context for all concepts can be found in the Key Concepts section of the Appendices.

Key capabilities for Camtek users include:

- Automated human-in-the-loop labeling
- Performance analysis and tracking
- Integration with the Camtek training infrastructure for real-time model deployment

The rest of this section describes:

- Key concepts
- Prerequisites for exploring datasets with Visual Layer

Key Concepts

Before exploring **Visual Layer**, familiarize yourself with these core concepts:

Concept	Description
Class	Each distinct type of label in your classification system. You need at least two classes, each with enough examples.
Class Taxonomy	The complete structure of all possible classes defined in classes.json from your AOI machine output. Cannot be changed after dataset creation.

Concept	Description
Cluster	A group of visually similar images automatically organized by **Visual Layer**
Confidence Score	A numerical measure (0-100%) indicating how certain the system is about an automatically assigned label.
Dataset	A collection of images and associated metadata organized for analysis and labeling
Embedding	The technology that enables **Visual Layer** to find visually similar images and automatically label thousands from just a few examples.
Ground Truth	Verified, accurate labels serving as the authoritative reference for training and evaluating machine learning models.
Label	The category assigned to a single image—the answer to "what type is this?"
Label Propagation	Semi-supervised learning combining automated pattern recognition with human oversight to efficiently label large datasets.
Seed	Initial labeled examples provided for each class. These training examples teach the system what each class looks like.

> **Need more detail?** For comprehensive definitions with examples and usage context, see the Key Concepts section in the **Appendices**.

Prerequisites

Chrome browser required.

SECTION II

The Workflow

The Workflow: Getting Started

This section guides you through the complete **Visual Layer** workflow for manufacturing defect detection, from dataset creation through model training.

The following diagram outlines the recommended sequence for using **Visual Layer** within a standard defect-detection workflow.

Figure 1: The Primary Workflow

The key steps of this workflow are as follows:

Step	Phase	Description
1	**Create & Prepare Dataset**	Ingest your production imagery and metadata from manufacturing equipment into the **Visual Layer** platform, then explore it to prepare for label propagation.
2	**Label Propagation**	Add representative samples, automatically propagate labels to similar images, and review and validate labels to ensure consistency and accuracy across your dataset.
3	**Train Models Downstream**	Send your finalized, validated dataset to your model-training pipeline for deployment.

Step 1: Create & Prepare Dataset

Create & Prepare: How It Works

Creating a dataset is the first main step in the **Visual Layer** workflow for curating your data and labeling images.

Camtek automated optical inspection (AOI) scans produce a standardized directory containing inspection results, production metadata, class/label definitions, and the associated images organized into subfolders. **Visual Layer** processes the output from Camtek AOI machines to rapidly train defect detection models.

> ⓘ Multiple folders must share identical Job/Setup/Recipe to be combined. This structure is required in order to ensure that in any given dataset, only outputs from the same production job are included.

You can quickly create and prepare a dataset for label propagation as outlined in the following diagram:

Figure 2: Dataset Creation & Preparation Workflow

Step	Phase	Description
1	**Upload & Select Folders**	Browse and upload AOI output directories that you want to include in the dataset. Upload as many as you'd like. For each folder, **Visual Layer** then automatically validates required files.
2	**Build & Create**	From the list of uploaded folders, checkmark all the folders to be included in the dataset. **Visual Layer** then creates the dataset, indexing and clustering images, and loading your class taxonomy based on the information that was included in each uploaded folder from Camtek.
3	**Explore Dataset**	When the dataset is ready, explore the dataset to identify representative examples for each defect class.

Create & Prepare: Upload & Build

This section guides you through creating datasets in **Visual Layer** from one or more AOI machine output directories derived from the same production job.

You'll:

- choose and upload output folders from your machines for Visual Layer validation
- select the folders relevant for one specific dataset
- confirm and wait for Visual Layer to generate the dataset

To create a dataset

1. From **Dataset Inventory** (the "Home" page) click **New Dataset.**

   The **Data** area loads.

2. From the **Enter Folder Paths** area, enter only the name of the relevant folder you want to upload.

   > The base path to your machine output is configured during installation. Enter only the folder name.

Figure 3: Create a New Dataset

3. You can upload additional folders at the same time, or one at a time. To add an additional folder, click **+**.

   An additional field appears for another folder name. Add as many as you need.

4. Click **Add** to add the folder/s to the list.

Visual Layer extracts metadata and validates folder structure and contents. When complete, the folders appear in the **Selected Folders** table.

Figure 4: Selected Folders table

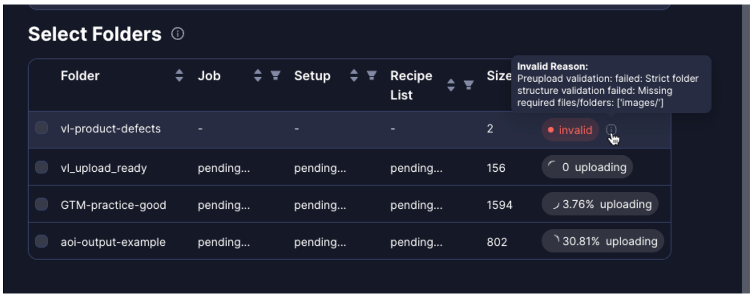
The table displays the following information for each folder:

Column	Description
Job Name	The production job identifier from the folder metadata
Setup Name	The setup configuration identifier from the folder metadata
Recipe List	The recipe name(s) used during inspection
Image Count	Total number of images contained in the folder

5. All folders in a dataset must share identical Job Name, Setup Name, and Recipe List. **If you uploaded multiple folders and need to identify which are compatible, use the table filters:**

   - **Filter by production metadata** - Use the Job Name, Setup Name, and Recipe Name filters to find folders from the same production run:

   - **Check compatibility** - Hover over the info icon to view details about any incompatible folders:



6. Select the folders to be included in the dataset that you're creating.

Once you select one folder from the list, **Visual Layer** validates compatibility across all folders to ensure they originate from the same production run. Folders with different metadata than the first folder you check are automatically disabled and grayed out. To see those folders again, uncheck any checked folders.

7. Click **Continue**.

   Visual Layer loads a preview of the intended dataset (a combination of all of the folders you selected) for you to review and approve.

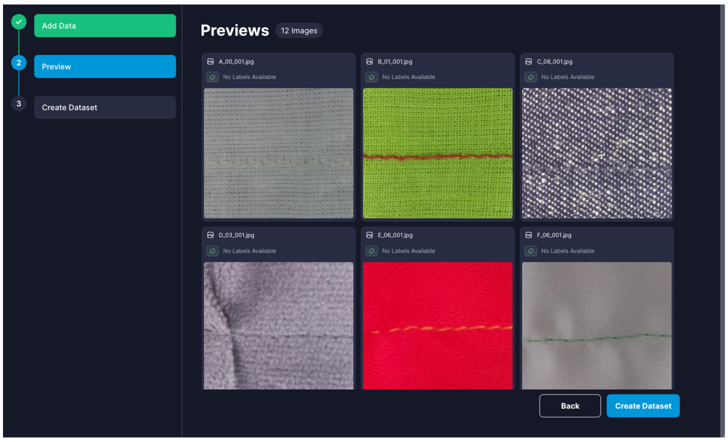


Figure 5: Preview Dataset

8. Click **Create Dataset**.

   Visual Layer begins processing the dataset.

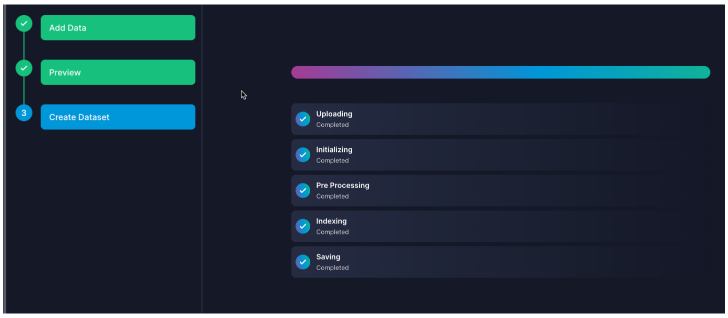


Figure 6: Creation Progress steps

Processing time varies with dataset size, typically 5-15 minutes for standard production runs. When complete, you'll receive a notification.

Create & Prepare: Explore

Visual Layer automatically organizes your images into clusters during dataset creation. Each cluster groups visually similar images together and displays as a preview card showing:

- **Number of items** - Total images in the cluster matching your current filters
- **Representative thumbnails** - Key example images that summarize the cluster's visual patterns
- **Prominent labels** - Most frequent labels within the cluster (if labels exist)

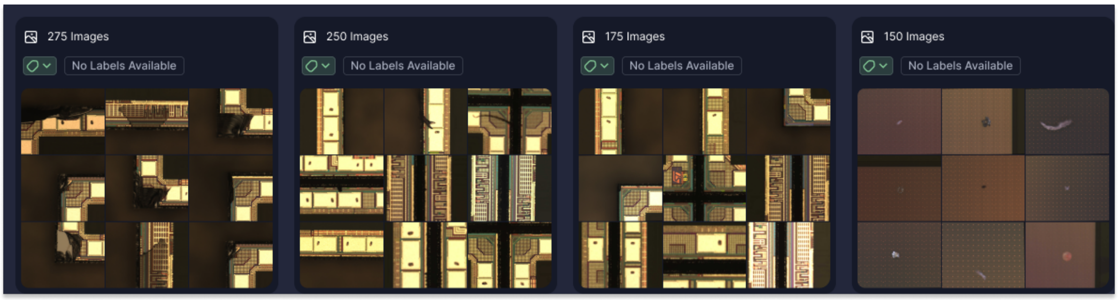


Figure 7: Cluster preview cards

Clusters help you quickly identify patterns and find similar defects without examining every image individually.

Once your dataset is ready, spend time exploring it before beginning label propagation to identify representative examples for each defect class. This exploration phase is critical for selecting high-quality seed examples that will train the system effectively.

> Spend sufficient time exploring before starting label propagation. The quality of your seed examples directly impacts automatic labeling accuracy. Look for clear, unambiguous examples that represent the full visual range of each defect class.

To explore and prepare for label propagation

1. From the **Dataset Inventory**, search or scroll to find your dataset and click its card to open it.

   The dataset loads in cluster view, displaying groups of visually similar images.

   Clusters help you quickly identify patterns and find similar defects without examining every image individually.

2. Look for clusters that clearly represent distinct defect types. Click any cluster to examine its images more closely and understand what visual patterns define each class.

3. Click to open the **Filter** options. For example, filter by **Folders** to examine images based on specific production runs.

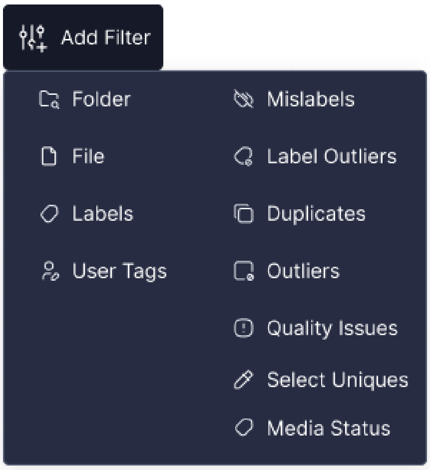


Figure 8: All Filters

4. When you find a good example of a defect class, hover over the image and click to find similar images. This helps you locate all potential candidates for that class.

5. Based on your exploration, take note which defect classes are well-represented in your data and where you can find at least 5 clear, diverse examples for each class.

Next Steps

Now that you've explored your dataset and identified representative examples for each defect class, you're ready to begin label propagation.

Step 2: Label Propagation

Label Propagation: How It Works

Label propagation is a semi-automated labeling process that transforms visual data annotation from a one-time task into a continuous improvement cycle. **Visual Layer** first receives instructions and human oversight to teach the system which labels to apply and for what kinds of images, combining automated pattern recognition with human oversight to create a feedback loop where each iteration improves labeling accuracy.

This feedback loop gives the system the guidance necessary in order to guarantee near-perfect automatic labeling thereafter.

ⓘ	Each dataset maintains its own independent model, preventing cross-contamination between production runs.

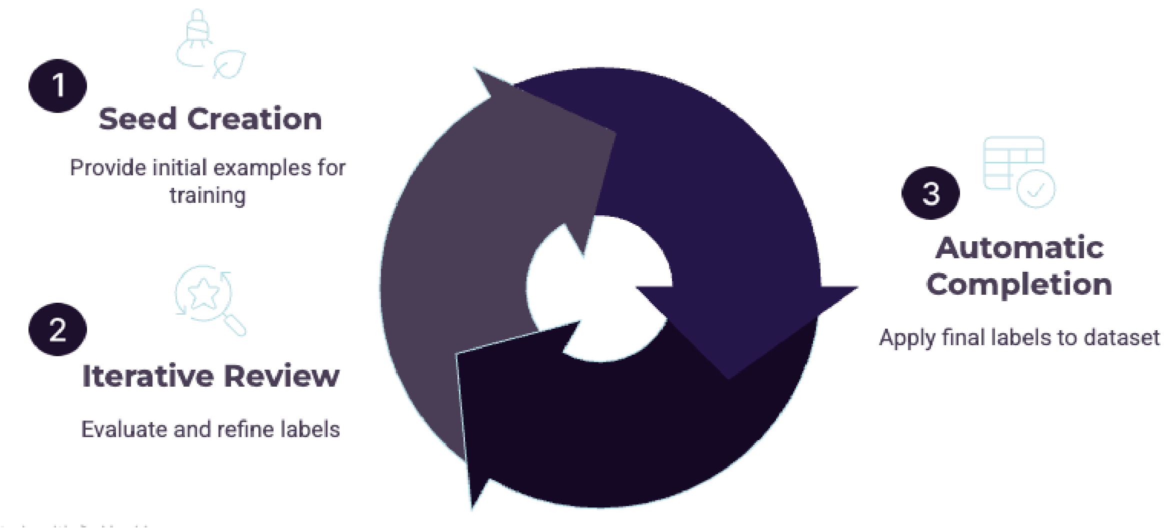


Figure 9: Label Propagation Workflow

The label propagation process includes these substeps:

Step	Phase	Description
1	**Seed Creation**	Provide ≥5 examples per class to teach the system what each class looks like.
2	**Iterative Review Cycle**	Work with **Visual Layer** to refine labels: • The system automatically labels high-confidence images • Uncertain images are sent to you in small review batches • You review and provide feedback on these batches • The system learns and improves with each review round
3	**Automatic Completion**	When model confidence reaches the threshold, label propagation automatically completes and applies final labels to the entire dataset.

Label Propagation: Add Seeds

To get started with label propagation you'll provide **Visual Layer** with seed examples that train our system to automatically label your remaining data. Label propagation can run on either the entire dataset or a subset, for example newly added data only.

> 1. **Visual Layer** assigns a single label to each image during label propagation. Multi-label classification is not supported.
> 2. Starting label propagation over from scratch **replaces** all previous label propagation results—existing labels are overwritten and cannot be recovered.
> 3. After finalizing label propagation, labels cannot be edited without running a new label propagation iteration.

> During seed creation, only class assignment is available and other exploration features are disabled.

Best practices

To maximize success with label propagation:

Best Practice	Description
Plan Your Classes	Select ≥2 classes. Include every class that may appear—classes cannot be added later.
Provide ≥5 Examples Per Class	Minimum required to proceed. More diverse examples = better accuracy.
Choose Representative Examples	Select clear, typical instances showing the full visual range of each class. Avoid edge cases or ambiguous images.
Triple-Check Before Starting	Seed examples cannot be edited once propagation begins. Verify all assignments are correct.

To prepare your dataset

1. Navigate to the relevant dataset in **Visual Layer**.
2. From the panel on the right side, click .

   The **Label Propagation** tab loads.

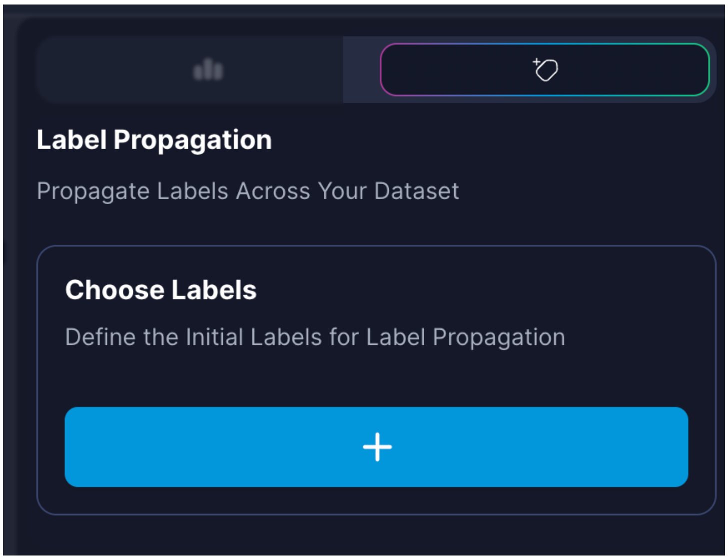


Figure 10: Label Propagation Tab

3. To start, you need to select all of the class labels that are relevant for the dataset.

   Click

   The **Add Labels** dialog pops open:

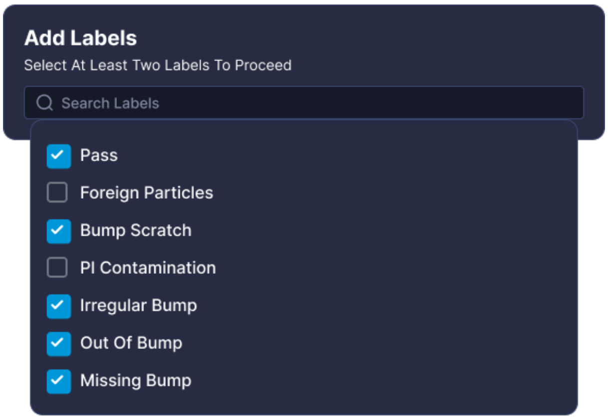


Figure 11: Add Labels dialog

4. Click in the field and choose labels from the dropdown list.

> You must select ***at least two classes*** for which you'd like to prepare labeling; classes not defined during this process cannot be assigned.

Select all classes that appear in your dataset. This is important because the system will attempt to label all data.

> This list contains the predefined classes from the `classes.json` file provided by your AOI machine and loaded during dataset creation.

5. Click **Add**.

   The label classes appear in the right panel.

6. For each class label, select a minimum of 5 seed examples:

   - **Drag and Drop** - For a single image or for a single cluster (in its entirety), drag it directly into the designated class area with visual feedback provided OR,
   - **Select & add** - Select all relevant images and then from the floating bar choose the relevant class and click **Assign Label**:

Figure 12: Assign Label button

Each class shows a preview of the examples that you assigned.

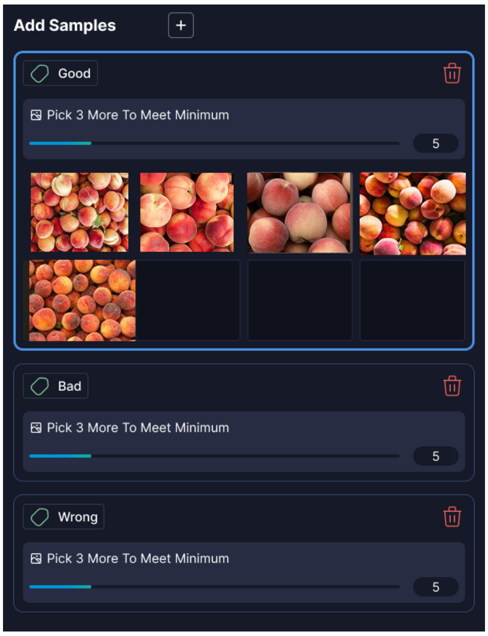


Figure 13: Seed Examples preview

7. Review and refine your selections:

   - To remove an example, hover over it and click .
   - Visual indicators show when you've met the minimum 5 examples per class requirement. See Progress Indicators in the **Appendices** for details on all progress tracking.
   - To manage classes as needed during the seed creation process, click **Add Labels** to add additional classes for propagation.
   - Click from the row with the class name to delete the entire sample set (and the label) from propagation.

8. When ready, click **Submit**.

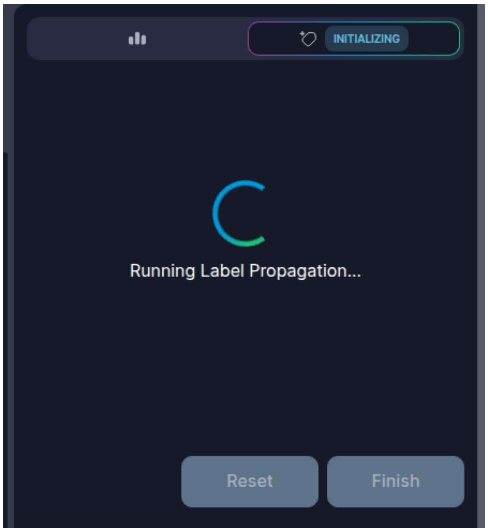


Figure 14: Label Propagation Running

Visual Layer analyzes your seed examples and begins labeling images, creating sample sets for each of the class labels you configured.

The system assigns confidence scores to each image for every possible label. Images with high confidence scores, typically 90% or higher, are automatically labeled (Annotated). Images with low confidence scores are sent to review batches for your input (For Review). When complete, a notification for review appears in the interface:

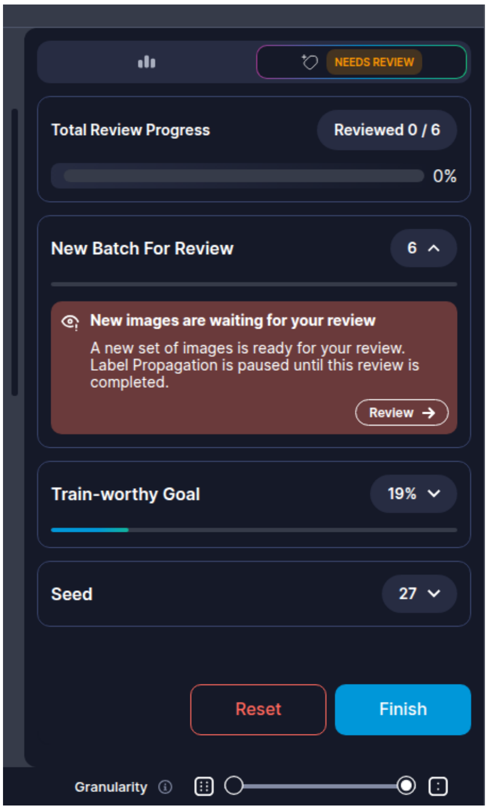


Figure 15: Review Notification

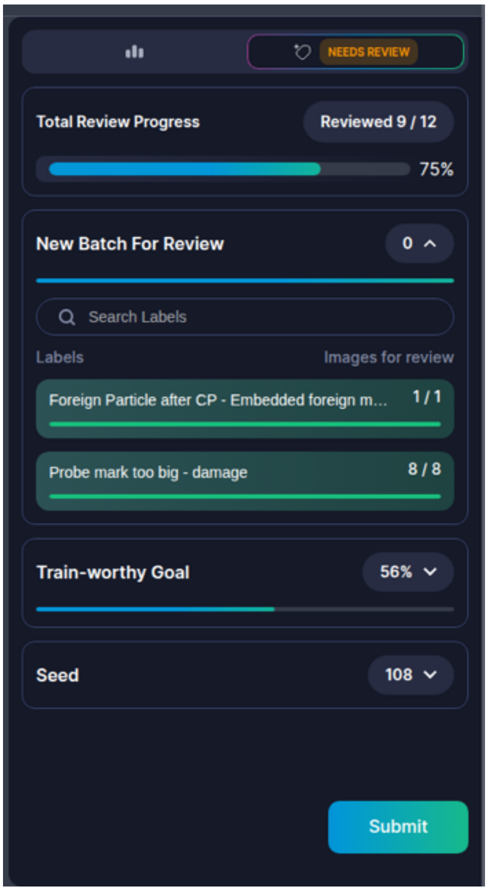


Figure 16: Progress & Review

Next Steps

When you receive notification, it's time to review the labels added by **Visual Layer**, and continue providing feedback to prepare the system for automated propagation.

Label Propagation: Review & Iterate

After submitting seeds, label propagation enters the iterative review cycle.

Visual Layer uses a confidence-based approach that creates review batches when it's uncertain about image classifications. This process ensures labeling accuracy while maintaining efficiency and continues until the model reaches sufficient confidence across your dataset. The system assigns confidence scores to each image for every possible class, creating targeted review batches for human verification.

Step	Phase	Description
1	**Confidence Scoring**	After analyzing your seed examples, **Visual Layer** assigns confidence scores (0–100%) to each image for every possible class.
2	**Automatic Labeling**	Images with high confidence scores (typically 90% or higher) are automatically labeled and do not require review.
3	**Review Batch Creation**	Images with low confidence scores are sent to review batches. Each review round includes a maximum of 5–10% of your total dataset. Example: with 1,000 images, you review approximately 50–100 images.
4	**Human Feedback**	Label propagation pauses and waits for your feedback. The process does not continue until the entire review batch is completed. Review automatically generated labels, especially items flagged for attention. You approve, correct, or reassign labels for uncertain images. Your feedback directly improves model accuracy in subsequent iterations.
5	**Refinement**	**Visual Layer** incorporates feedback and relabels the dataset with improved accuracy. This cycle repeats, generating new review batches as the model refines its understanding.

Source Indicators

Icon	Source	Description
VL	**VL**	Label assigned by **Visual Layer** with high confidence
User	**User**	Label reviewed and confirmed by you in previous iterations
(no icon)	**Seed**	Original examples you provided during seed creation; **seed** is separated from **user** because seed-assigned labels cannot be edited later

> All VL and User items remain editable regardless of source. Previous review decisions can be corrected if needed.

Feedback Actions

During review rounds, you can provide feedback to the system via the feedback actions:

Figure 17: Feedback Options

Following are the feedback actions and how to use each:

	Action	Use Case	What Happens	Important Notes
	Incorrect (Reject)	Certain it's wrong class, unsure of which label is correct	Item moves to next-best predicted class Remains flagged "Requires Review" Can continue rejecting until right class found	Undo available via toast notification Example: Unknown defect types cycle through suggestions until recognized
	Ignore	Outlier that doesn't match any existing class	Item excluded from label propagation Placed in "Unlabeled" cluster Appears grayed out Does not count toward progress	System maintains 5–10% review quota Too many ignored items may trigger additional review batches
	Approve	Item correctly matches assigned class	Green border indicates approval Advances to next item	Confirms model prediction
	Assign to Other Class	Assign to a different existing class	Manually reassigned to selected class Advances to next item	Directly teaches model correct classification

> 1. Items are organized by their suggested class based on highest confidence score.
> 2. You can provide feedback on both review items and high-confidence items (marked Annotated).
> 3. All feedback helps improve subsequent labeling iterations.

Progress Indicators

Throughout the label propagation process, **Visual Layer** displays progress indicators that track your advancement toward training readiness (minimum 100 labeled images per class) and overall dataset completion. These indicators help you understand when you've met the requirements to train models and make informed decisions about when to finish label propagation.

See Progress Indicators in the **Appendices** for detailed descriptions and examples of each progress indicator.

To start a review round

1. When a review batch is ready for your review, a notification appears in the interface:

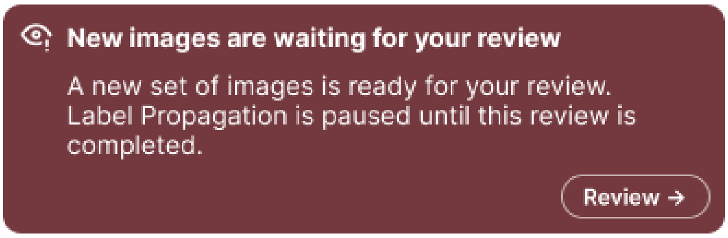


Figure 18: Review Batch notification

The labeled images are automatically grouped by predicted class/label.

2. Click **Review** to begin reviewing images that were automatically labeled by **Visual Layer**.

ⓘ	During review, all exploration features like filters, search, and visual similarity are disabled.

Visual Layer automatically loads one of the clustered classes for review. The view displays all images that were assigned the same predicted label, grouped together for efficient review:

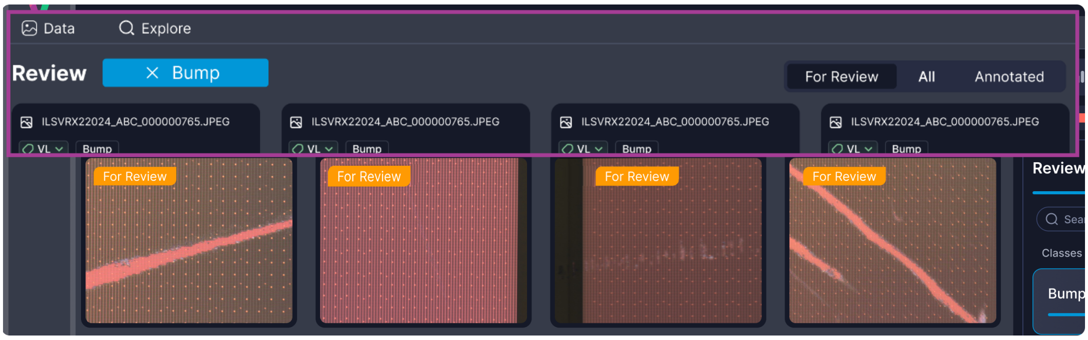


Figure 19: Review Round - First Class Loaded

You can search for specific classes if you have many classes to review:

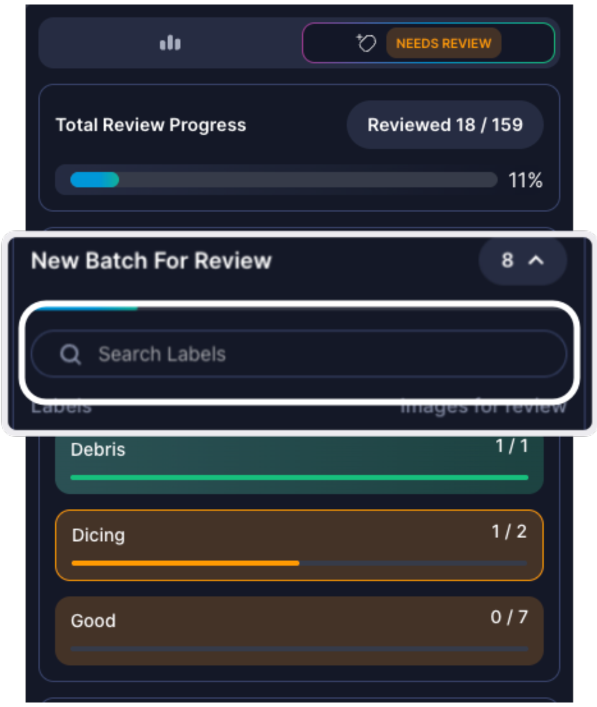


Figure 20: Search classes

Within each class, you can switch between two tabs:

- **For Review** (default tab) - Displays images that need your review, marked with For Review.
- **Annotated** - Displays seed samples you provided and images that **Visual Layer** labeled with high confidence (typically ≥90%). All items are marked with Annotated and show their source indicator (VL for automated labels, User for reviewed items, or Seed for original examples).

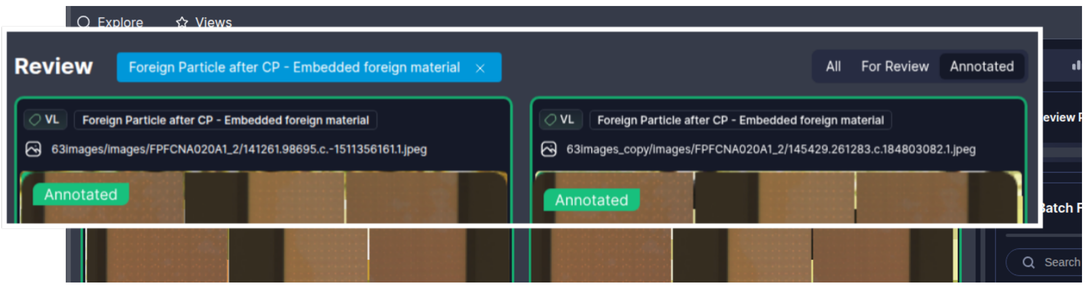


Figure 21: Annotated Tab View

You can also view all clusters included in the propagation, similar to other cluster views in **Visual Layer**. Each cluster card displays:

- Total number of images in the cluster for this review round (including both Annotated and For Review images)

- Number of **For Review** images specifically for that cluster

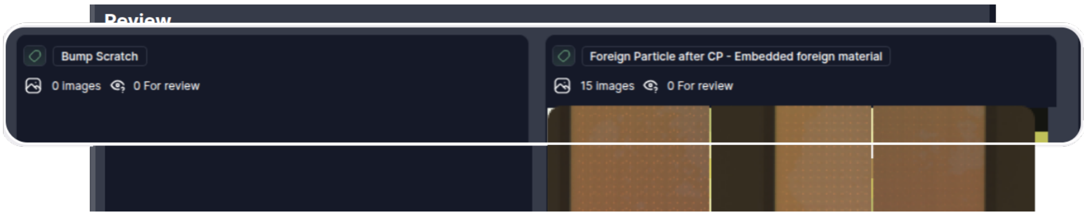


Figure 22: All Clusters View

> If a class has no images for review in the current round, its cluster card appears but shows no content, as seen with "Bump Scratch" in the example above.

3. Review every image for every class:
   - Provide feedback directly from the default display.
   - Alternatively, double-click any image to open a modal carousel view.

Figure 23: Carousel Modal

   - Navigate through individual items using carousel controls.
   - Zoom in/out for detailed defect inspection.
   - Click a label from the right side to apply a different label than was initially applied.
4. For every automatically labeled image, provide the model with feedback:
   - Approve
   - Reject
   - Ignore
   - Apply different label

See Feedback Actions for more information.

5. Review each item in every class until all items are processed.

6. Click **Submit** to send all your feedback to the system. Label propagation continues with the next iteration, using your feedback to improve accuracy in subsequent labeling cycles. When the system has high confidence, it automatically applies labels during the relevant cycle.

7. Once the system is confident and applies all labels across your entire dataset, a notification pops up, similar to the following:

Figure 24: Propagation Success

Managing Iterations and Completion

This section outlines how to manage the lifecycle of a label propagation run after seed creation and initial labeling:

- finishing the run early to finalize labels
- resetting the process if the results are unsatisfactory

Finish Label Propagation Early

Once the system is confident and applies all labels across your entire dataset, a notification pops up.

You can finish the process early, enabling Visual Layer to apply labels across your dataset before completing all iterations. This is available when **Needs Review** status is active only.

From the review details in the right panel:

1. Click **Finish** when satisfied with current progress.

2. Confirm the action in the dialog that appears.

   The system:

   - Stops label propagation immediately
   - Writes all labeled data permanently to your dataset
   - Leaves remaining unlabeled data unchanged

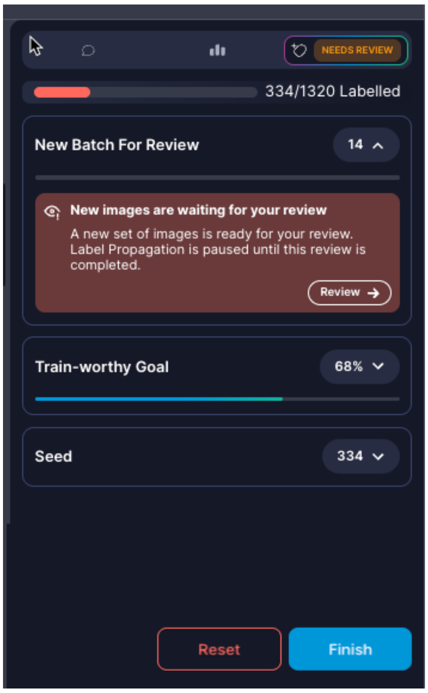


Figure 25: Finish Early

Resetting the Process

You can reset the process only during **Review** stages. Reset ***permanently deletes*** all label propagation progress and cannot be undone. Your original dataset and any labels from previous completed label propagation runs remain unchanged.

Reset label propagation if you:

- notice consistent labeling errors that suggest poor seed examples
- want to try a different class taxonomy or seed strategy
- the results are not meeting your accuracy expectations
- accidentally started with incorrect seed examples

To reset the process

> ⚠ Reset ***permanently deletes*** all label propagation progress and cannot be undone. Your original dataset and any labels from previous completed label propagation runs remain unchanged.

1. From the right panel, click **Reset**.
2. Confirm the reset action in the warning dialog.
3. The system completely deletes all:
   - automatically labeled images (high-confidence assignments)

- user review feedback and corrections
- progress and iteration history

Next Steps

When your dataset reaches train-worthy status, it's time to train your downstream models. Alternatively, you can evaluate labeling results.

Step 3: Train Models Downstream

Train Models: How It Works

Training models downstream is the last step in your primary **Visual Layer** workflow. You can train a model downstream when the dataset has achieved **Train-Worthy** status, allowing you to send the complete dataset downstream to train your Camtek models. This integrated training workflow eliminates manual data export and preparation steps.

Visual Layer automatically formats your labeled dataset and sends it to the Camtek training infrastructure with a single click. When sending the data for training, the system uses all labeled data from your completed label propagation, including seed examples, reviewed items, and high-confidence automatic labels. No manual data preparation is required.

ⓘ	You can use a single dataset to support the training of multiple models over time; however, you can only run one training session per dataset at a time.

The downstream model training process consists of the following substeps:

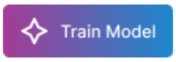
Figure 26: Train Models Downstream Workflow

Step	Phase	Description
1	**Initiate Training**	Select your target Camtek model configuration and click **Train** to package and send your labeled dataset to the training infrastructure.
2	**Monitor Progress**	Track training job status through the **Visual Layer** interface. View real-time updates as your model trains on the downstream infrastructure.
3	**Review Results**	Access completed models in the Model Catalog to view training results, performance metrics, and deployment status.

Train Models: Initiate

Once your dataset reaches train-worthy status with at least 100 labeled images per class, you can initiate model training to send your labeled data to the Camtek training infrastructure.

To initiate model training

1. Navigate to the relevant dataset.
2. Click 

in the top right corner. The **New Auto Training Job** dialog opens, similar to the following:

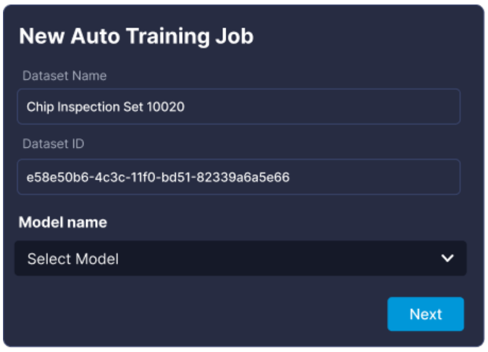


Figure 27: Training Job dialog

Following are details of the fields in this window:

Component	Description	Example/Notes
Dataset Name	The human-readable name of the **Visual Layer** dataset being sent for training.	Populated automatically from the current dataset. (Read-only, for reference); you can change the name from the title bar inside the dataset itself.
Dataset ID	The unique system identifier for the dataset.	For example: 0f4698-9940-11f0-8218-6ad3b2ffe3e5 (Read-only, for reference)

Component	Description	Example/Notes
Model Name	Allows selection of the target Camtek model configuration for this training run. The available models are automatically retrieved from your Camtek training infrastructure.	Select from the dropdown list of models configured in your Camtek environment.
Train button	Initiates the training job.	Click to package the labeled data and send it to the Camtek training infrastructure.

3. From **Model Name** choose the relevant model.
4. Click **Train**.

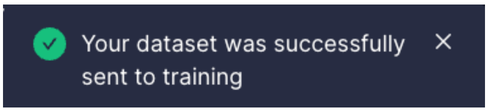


Figure 28: Training Success

Visual Layer automatically packages your labeled dataset with all metadata and sends it to your training infrastructure.

5. Monitor the training progress, which appears in the action bar:

Figure 29: Training Progress indicator

Visual Layer provides real-time progress updates showing the current status and completion percentage (0-100%). If training fails, hover over the status to view error details. See Model Training Statuses in the Appendices for descriptions of all training stages.

Training typically completes within several hours depending on dataset size and model complexity.

After training completes

When model training finishes successfully, Camtek returns the HTML report path and **Visual Layer** marks the model as **Completed** in the **Model Catalog**. A success notification appears: **Your model has been created successfully. You can explore it in the Model Catalog.**

Click **View Model** to navigate directly to the **Model Catalog** where your new model will be highlighted and auto-scrolled into view.

Train Models: Model Catalog

The **Model Catalog** is the central hub for managing your downstream model training and results. Once an external model training job successfully completes, the trained model data is accessible from the **Model Catalog**. From here, you can:

- View detailed training results in HTML format
- Export the entire model results folder as a ZIP file
- Delete models from the **Visual Layer** catalog

To get started

From the left of the **Visual Layer** interface, click .

The **Model Catalog** loads, and appears similar to the following:

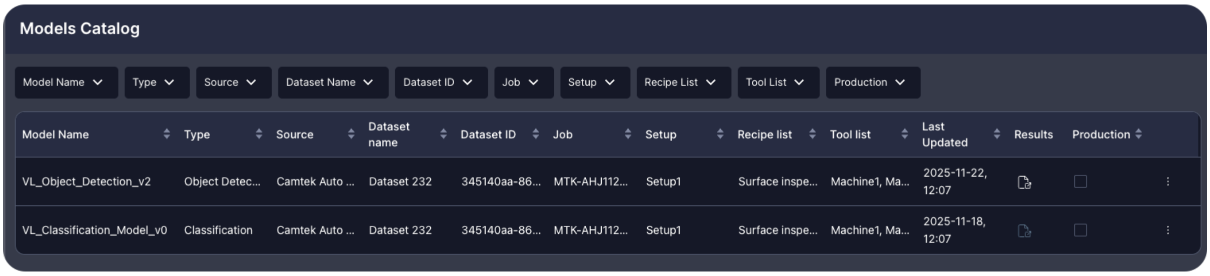


Figure 30: Model Catalog

Catalog Structure and Features

The **Model Catalog** displays a table view of all the models you've sent data to for training. All columns support filtering and ascending/descending sorting for easy navigation.

The table displays the following information:

Column	Description
Model Name	The name of the selected model, as returned from Camtek.
Type	The type of the model. This is currently always **Classification**.
Source	The training source. This is currently always Camtek Auto ML.
Dataset Name	The human-readable name of the dataset in **Visual Layer** used for training.
Dataset ID	The unique system identifier of the training dataset in **Visual Layer**.
Job	The production job identifier for the training job.
Setup	The machine setup configuration used.
Recipe List	A list of the scanning recipes applied.
Tool List	The tools used.
Last Updated	The timestamp from the training job results folder.
Results	Click the icon to view the HTML report. The report is embedded within **Visual Layer** for easy access.
Production	Check or uncheck to mark models as production-ready. Unchecking requires confirmation: "Confirm removal from production - You are about to unmark this model as production. Are you sure?"
Three-Dot Menu	Provides key functions for managing trained models: **Export** downloads the entire model results folder as a ZIP file to your local Downloads folder. **Delete** removes the model entry from the catalog (requires confirmation, elevated users only).

To remove a model from production

1. Uncheck the **Production** checkbox for the relevant model.

   A confirmation modal appears:

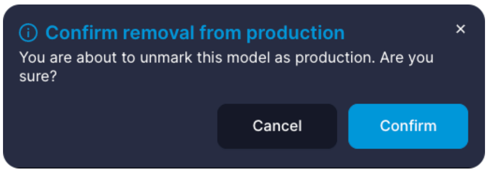


Figure 31: Remove from Production Confirmation

2. Click **Yes** to confirm the removal (or **Cancel** to keep the production status).

   The model is unmarked as production.

To delete a model from the catalog

1. From the relevant row in the **Model Catalog**, click the three-dot menu and choose **Delete**.

   A confirmation modal appears:

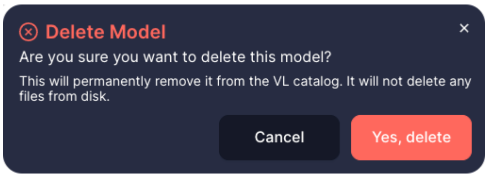


Figure 32: Delete Model Confirmation

2. Click **Yes, delete** to confirm (or **Cancel** to stop).

   The model entry is immediately removed from the catalog interface.

> Deleting a model from the catalog does not delete the model files from disk. The files remain in their original location.

Understanding Model Training Results

When model training completes successfully, Camtek generates comprehensive validation reports that assess your trained model's classification performance. These reports help you evaluate model accuracy, identify areas for improvement, and make informed decisions about deploying models to production.

The training results package includes two main HTML reports along with supporting data files:

- **Validation Summary** - Overview of model performance with confusion matrix and per-class metrics
- **Sample Metrics Report** - Detailed per-image uncertainty and confidence metrics
- Supporting CSV files with predictions, logits, and confusion matrix data
- Transformed debug images showing model processing

To access training results

1. To view the **Validation Summary** report from your browser, click in the **Results** column.

   OR

   To export results, click the three-dot menu for the relevant model and choose **Export**.

   The entire results folder downloads as a ZIP file to your Downloads folder.

2. Extract the ZIP file.

   The extracted folder contains:

   - `summary/validation_summary.html` - Main validation report
   - `sample_metrics_report/report.html` - Detailed metrics report
   - `summary/predictions.csv` - Image-by-image predictions
   - `summary/confusion_matrix.csv` - Classification confusion matrix
   - `summary/logits.csv` - Raw model output scores
   - `summary/missed_images_data.js` - Misclassified samples data
   - `summary/images_debug_transformed/` - Processed image directory
   - `sample_metrics_report/images/` - Original images directory
   - `js/` and `style/` - Required JavaScript and CSS files

3. Double-click the `html` file for the relevant report to open it in your browser.

> Save the extracted results folder for your records. The reports remain fully functional offline.

Validation Summary Report

The **Validation Summary** report provides a comprehensive overview of your trained model's performance on the validation dataset.

General Information

The report header displays key metadata about the training run:

Fields include:

Figure 33: Training Run Metadata

Field	**Description**
Tag	Identifier for this training run and model configuration
Generated on	Date and time when the validation report was created (YYYY-MM-DD HH:MM:SS format)
Data path	Location of the validation dataset used to evaluate model performance
Checkpoint	Path(s) to the trained model checkpoint file(s)
Dataset size	Total number of images in the validation set and the split used
FPR@95%TPR (OOD)	False Positive Rate at 95% True Positive Rate for out-of-distribution detection

Class Distributions by Zone

An expandable histogram section showing how samples are distributed across different inspection zones and classes.

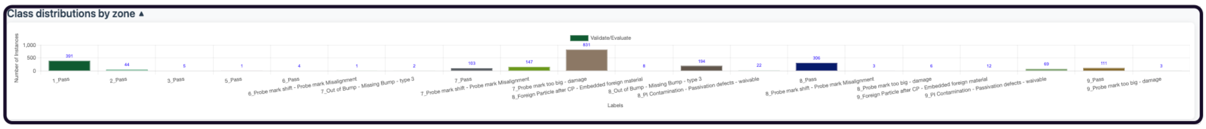
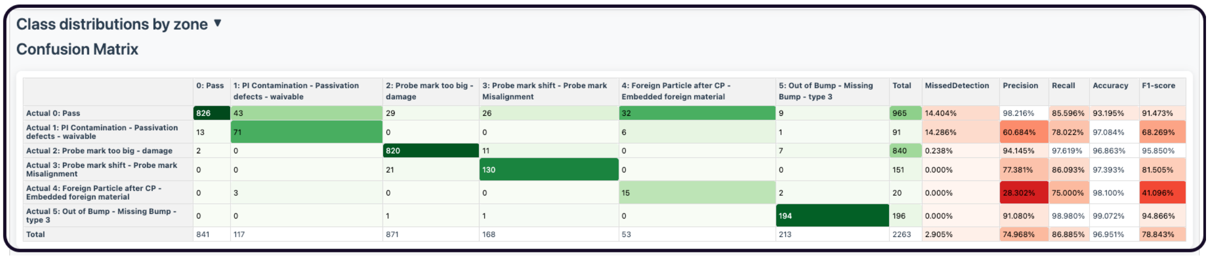


Figure 34: Class Distribution by Zone

This helps identify data imbalances that may affect model performance.

Confusion Matrix

The confusion matrix displays actual ground truth classes versus predicted classes, with color coding to highlight classification performance:



Class distributions by zone ▼

Confusion Matrix

	0: Pass	1: PI Contamination - Passivation defects - waivable	2: Probe mark too big - damage	3: Probe mark shift - Probe mark Misalignment	4: Foreign Particle after CP - Embedded foreign material	5: Out of Bump - Missing Bump - type 3	Total	MissedDetection	Precision	Recall	Accuracy	F1-score
Actual 0: Pass	826	43	29	26	32	9	965	14.404%	98.216%	85.596%	93.195%	91.473%
Actual 1: PI Contamination - Passivation defects - waivable	13	71	0	0	6	1	91	14.286%	60.684%	78.022%	97.084%	68.269%
Actual 2: Probe mark too big - damage	2	0	820	11	0	7	840	0.238%	94.145%	97.619%	96.863%	95.850%
Actual 3: Probe mark shift - Probe mark Misalignment	0	0	21	130	0	0	151	0.000%	77.381%	86.093%	97.393%	81.505%
Actual 4: Foreign Particle after CP - Embedded foreign material	0	3	0	0	15	2	20	0.000%	28.302%	75.000%	98.100%	41.096%
Actual 5: Out of Bump - Missing Bump - type 3	0	0	1	1	0	194	196	0.000%	91.080%	98.980%	99.072%	94.866%
Total	841	117	871	168	53	213	2263	2.905%	74.968%	86.885%	96.951%	78.843%

Figure 35: Confusion Matrix

Rows represent actual ground truth labels, while columns show predicted labels. The matrix includes performance metrics:

Metric	Description
FP Rate	False Positive Rate - percentage of negative samples incorrectly classified as this class
Precision	Percentage of predictions for this class that were correct
Recall	Percentage of actual instances of this class that were correctly identified
Accuracy	Overall percentage of correct classifications for this class
F1-Score	Balanced measure combining Precision and Recall

The diagonal cells (where actual equals predicted) represent correct classifications and are highlighted with darker green shading. Off-diagonal cells show misclassifications with lighter shading or red tones.

Defects Data Table

The detailed defects table displays every image in the validation set with its actual and predicted classifications.

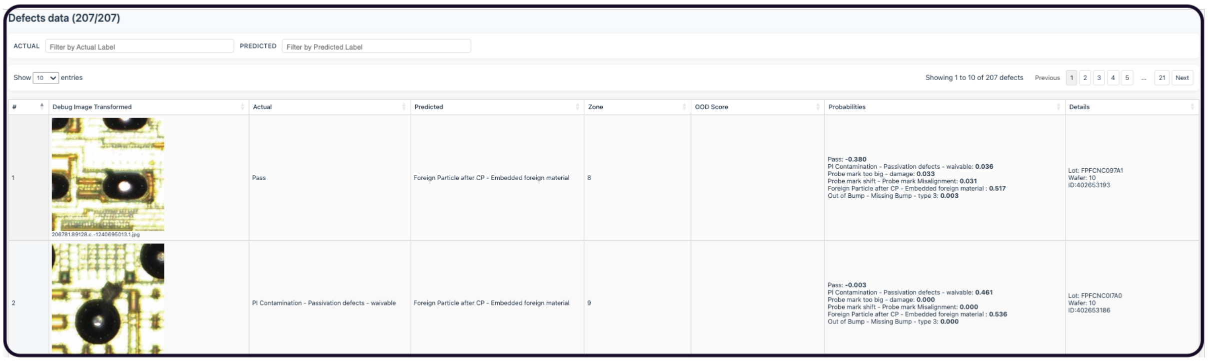


Figure 36: Confusion Matrix

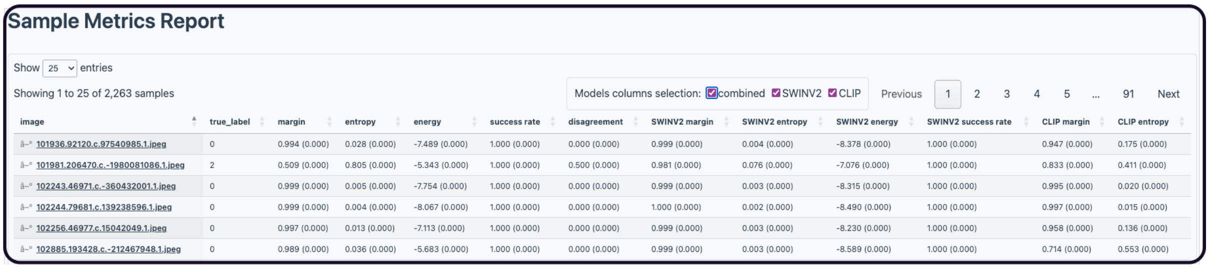
Use the dropdown filters to

- **Actual** - Filter by ground truth class
- **Predicted** - Filter by model prediction

This table helps you identify specific misclassification patterns and review individual model decisions.

Sample Metrics Report

The **Sample Metrics** report provides detailed uncertainty and confidence metrics for each image in the validation set.



Sample Metrics Report

Show 25 entries

Showing 1 to 25 of 2,263 samples

Models columns selection: combined SWINV2 CLIP

Previous 1 2 3 4 5 ... 91 Next

image	true_label	margin	entropy	energy	success rate	disagreement	SWINV2 margin	SWINV2 entropy	SWINV2 energy	SWINV2 success rate	CLIP margin	CLIP entropy
â–° 101936.92120.c.97540985.1.jpeg	0	0.994 (0.000)	0.028 (0.000)	-7.489 (0.000)	1.000 (0.000)	0.000 (0.000)	0.999 (0.000)	0.004 (0.000)	-8.378 (0.000)	1.000 (0.000)	0.947 (0.000)	0.175 (0.000)
â–° 101981.206470.c.-1980081086.1.jpeg	2	0.509 (0.000)	0.805 (0.000)	-5.343 (0.000)	1.000 (0.000)	0.500 (0.000)	0.981 (0.000)	0.076 (0.000)	-7.076 (0.000)	1.000 (0.000)	0.833 (0.000)	0.411 (0.000)
â–° 102243.46971.c.-360432001.1.jpeg	0	0.999 (0.000)	0.005 (0.000)	-7.754 (0.000)	1.000 (0.000)	0.000 (0.000)	0.999 (0.000)	0.003 (0.000)	-8.315 (0.000)	1.000 (0.000)	0.995 (0.000)	0.020 (0.000)
â–° 102244.79681.c.139238596.1.jpeg	0	0.999 (0.000)	0.004 (0.000)	-8.067 (0.000)	1.000 (0.000)	0.000 (0.000)	1.000 (0.000)	0.002 (0.000)	-8.490 (0.000)	1.000 (0.000)	0.997 (0.000)	0.015 (0.000)
â–° 102256.46977.c.15042049.1.jpeg	0	0.997 (0.000)	0.013 (0.000)	-7.113 (0.000)	1.000 (0.000)	0.000 (0.000)	0.999 (0.000)	0.003 (0.000)	-8.230 (0.000)	1.000 (0.000)	0.958 (0.000)	0.136 (0.000)
â–° 102885.193428.c.-212467948.1.jpeg	0	0.989 (0.000)	0.036 (0.000)	-5.683 (0.000)	1.000 (0.000)	0.000 (0.000)	0.999 (0.000)	0.003 (0.000)	-8.589 (0.000)	1.000 (0.000)	0.714 (0.000)	0.553 (0.000)

Figure 37: Sample Metrics report

This advanced report helps you:

- Identify images where the model is uncertain
- Understand prediction confidence levels
- Compare performance between different model architectures (SWINV2 vs CLIP)
- Detect potential data quality issues

The report includes per-image metrics:

Metric	Description
Margin	Difference between top prediction and second-best prediction (higher = more confident)
Entropy	Measure of prediction uncertainty (lower = more confident)
Energy	Alternative confidence measure based on logits
Predicted Ratio	Ratio of top prediction probability
Ensemble Disagreement	Disagreement between multiple model architectures

Metrics are provided for:

- **Combined** - Ensemble metrics across all models

- **SWINV2** - Vision transformer model metrics
- **CLIP** - Multimodal model metrics

Each metric includes both average and standard deviation values to capture prediction stability.

SECTION III

Advanced Dataset Features

Add Media to Datasets

As production continues, inspection machines continuously generate new data. New images may contain defects you're already tracking, introduce entirely new defect types, or present a combination of both.

You can add these images to existing datasets without creating new ones. This maintains dataset continuity across production runs and preserves your existing class taxonomy. After re-indexing, you can re-run label propagation on the complete dataset, reusing your proven seed examples while Visual Layer learns from both existing and newly added images together.

Adding media to an existing dataset enables you to:

- Expand training datasets with new production examples without fragmenting data across multiple datasets
- Maintain consistent Job Recipe and class taxonomy across all images in a dataset
- Continue label propagation workflows on newly added images using existing seed examples and model learning
- Track dataset evolution over time while preserving the original dataset ID and configuration

How It Works

Visual Layer uses a two-stage process when adding media to optimize performance and give you control over when computationally intensive operations run. New media becomes immediately visible for exploration after upload. You'll trigger indexing **separately** when you're ready.

Here's how it works:

Step	Phase	Description
1	**Upload and Validate**	Upload new AOI output folders. **Visual Layer** validates that the metadata matches the existing dataset exactly.
2	**Partial Update**	**Visual Layer** automatically generates embeddings for visual search, creates thumbnails, processes metadata, and groups new images into a temporary "New Media" cluster visible immediately for exploration. This cluster is part of your existing dataset, but separate from all other clusters during this stage, enabling you to review the added media before re-indexing completely. Additionally, you can repeat these steps to continue adding additional folders if you didn't include all relevant media initially. When this update is complete, all media that was newly added is in **Pending Indexed** status.
3	**Manually Trigger Re-indexing**	When ready, you can trigger full indexing. During indexing, **Label Propagation** is unavailable. **Visual Layer** re-clusters all images (new and existing), regenerates quality issue detection, and updates cluster IDs. Dataset status changes to **Indexing**.
4	**Ready for Use**	Indexing completes and dataset status returns to **Ready** status. All media in the datset is set to **Indexed** status. New images are fully integrated in the dataset, with all clusters refreshed accordingly, and available for label propagation and training.

> Dataset ID and all image IDs remain unchanged during indexing. Only cluster IDs are updated to reflect the new clustering results.

Adding Media to an Existing Dataset

Visual Layer uses a two-stage process when adding media to optimize performance and give you control over when computationally intensive operations run. New media becomes immediately visible for exploration after upload, with full indexing triggered manually when you're ready. You can upload multiple batches and explore before triggering the clustering and analysis steps.

> New media must match the metadata of the original dataset. Mismatched metadata will cause validation failures and prevent media from being added.
>
> To work with datasets that have different metadata, create a new dataset instead.

> If you encounter validation errors, see Troubleshooting Common Issues in the **Appendices** for required file specifications and common error solutions.

To add media to an existing dataset

1. Navigate to the dataset where you want to add new images.
2. Go to the **Data** tab.

   The **Data** area loads, similar to the following:

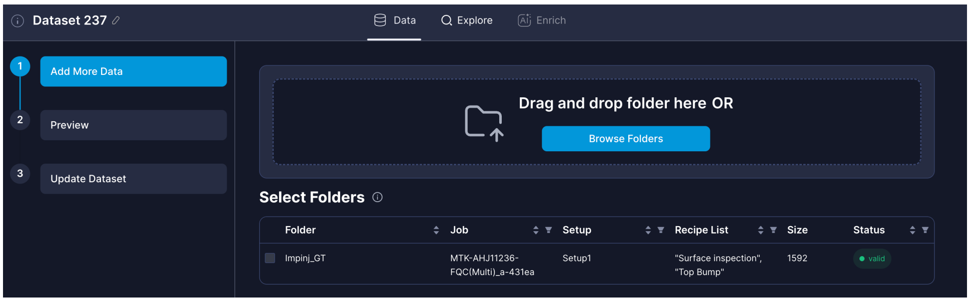


Figure 38: Data Tab for Adding Media

This tab uses the same interface as dataset creation, but adds media to the current dataset instead of creating a new one. The original folder/s you used to create the dataset appear in the **Select Folders** table, and you can add additional folders as well.

The name of the dataset you're updating appears at the top left accordingly. Hover over the ⓘ next to the dataset name to confirm its details:

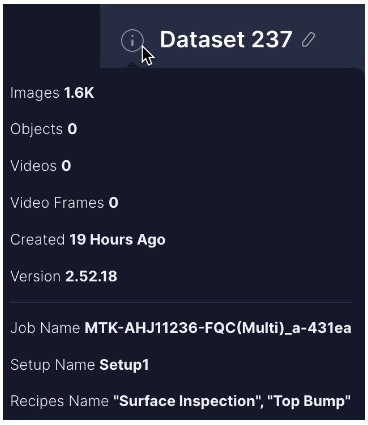


Figure 39: Dataset Details Tooltip

3. Click **Browse Folders** and select your AOI output folder containing the new images and metadata files.

   > You can add multiple folders to a single dataset, as described in the next step.

4. **Visual Layer** extracts metadata and validates folder structure and contents.

   When complete, the folder appears in a table on the same screen.

5. Upload additional folders that share identical metadata with the original dataset.

   > If you have multiple production runs to add, upload them all before triggering indexing. This reduces processing time and ensures the clustering algorithm analyzes your complete dataset together.

   Visual Layer validates Job/Setup/Recipe compatibility against the existing dataset to ensure all folders originate from the same production run.

Figure 40: Selected Folders table

6. Folders appear in the table showing their compatibility status:

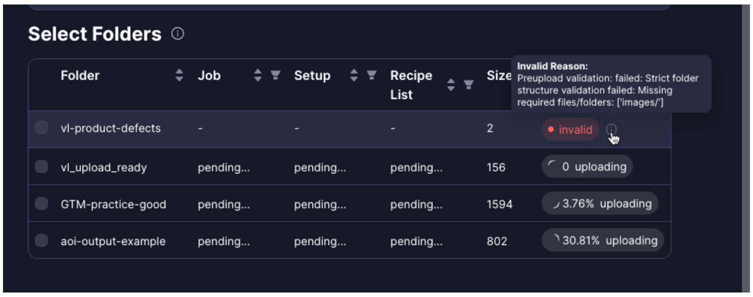
   > See Creating a Dataset for additional details.

7. Filter the **Selected Folders** table to identify compatible folders:

   Since you may not know the folder origins in advance, upload as many folders as relevant and then use the table filters to identify those that share identical Job/Setup/Recipe metadata with the existing dataset.

   - **Filter by production metadata** - Use the Job Name, Setup Name, and Recipe Name filters to identify folders from the same production run:

   - **Check compatibility** - Hover over the info icon to view details about any incompatible folders:



Compatible folders remain selectable, while incompatible folders (those with different metadata) are grayed out and cannot be selected.

8. Mark the folders from the list that should be added to the dataset and click **Continue**.

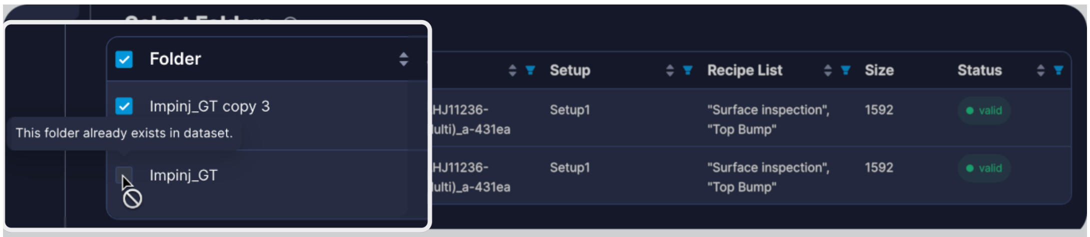


Figure 41: Selected Folders for Adding Media

Visual Layer loads a preview of the intended dataset update for you to double-check and approve.

9. Click **Add Media**.

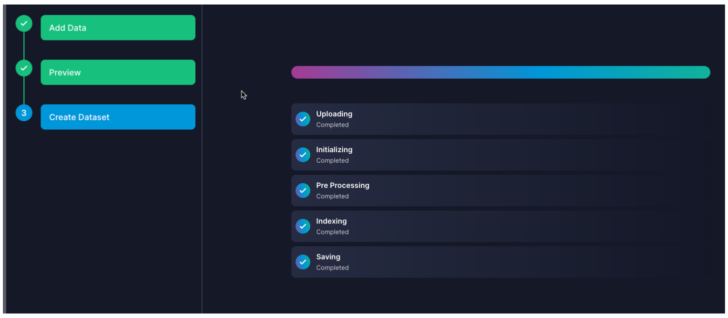


Figure 42: Upload Progress steps

10. When upload completes, a notification confirms that new media was successfully added. All newly added media is marked with status **Pending Indexed**, for which you can filter when exploring your dataset:

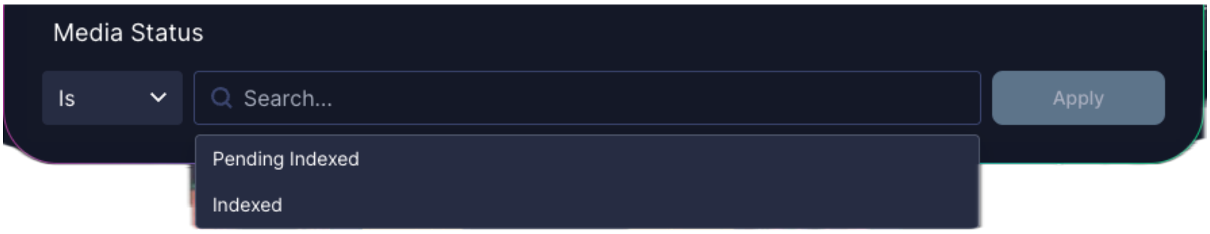


Figure 43: Pending Indexed

Re-indexing the Updated Dataset

After reviewing your new clusters, you can re-index the entire dataset in order to commence a new label propagation round.

1. When ready to proceed with label propagation or training, click **Re-Index Dataset**. If you plan to add more media, wait and batch multiple uploads before indexing.

   Full indexing begins. The dataset status changes to **Indexing**, and the following features become temporarily unavailable: **Label Propagation** and **Adding Media**.

   > ⓘ Track indexing progress in the **Task Manager**, which shows task type, dataset name and ID, duration, and status.

2. When indexing finishes, a notification appears confirming the dataset is ready.

   All features are re-enabled. Your newly added images are now fully integrated and available for label propagation and training.

Updating Dataset Labels

Once the dataset has been reindexed, you can return to label propagation. Assuming the labels you initially used are still relevant for images in the dataset, we recommend you approach this as follows:

1. From inside the relevant dataset, initiate label propagation, adding all of the relevant class labels.

   This includes any new labels that may now be relevant!

2. Filter for all of your original seed images for one of the relevant labels.
3. Select one, and then choose **Select All**.
4. Add them to the relevant label in the right panel.
5. Repeat the process for all other labels that you already identified seed examples for.

Evaluate Label Propagation

After completing label propagation, you can validate the accuracy of the automatically generated labels. The **LP Auto Annotation Validation Report** is an interactive HTML report comparing your Ground Truth data with the labels generated through the label propagation process.

Use the evaluation report to make informed decisions about your label propagation quality:

- Measure overall accuracy to determine if your dataset is ready for model training
- Identify which defect classes need additional seed examples or clearer definitions based on per-class precision
- Track error sources by label origin to determine whether additional review iterations or annotation guideline clarification is needed
- Locate classes with high missed detection rates requiring more diverse seed examples
- Export validation data for compliance documentation and process audits

Prerequisites

Before generating an evaluation report, ensure you have:

- Completed the label propagation feedback loop for the dataset
- A Ground Truth file (`db.csv`) containing validated annotations for your images
- A class mapping file (`classes.json`) defining the taxonomy used in your Ground Truth data

> The Ground Truth taxonomy must align with the classes used during label propagation. Additionally, in this file, the Image Name column which includes the relative path must include the folder name in which the image is saved.

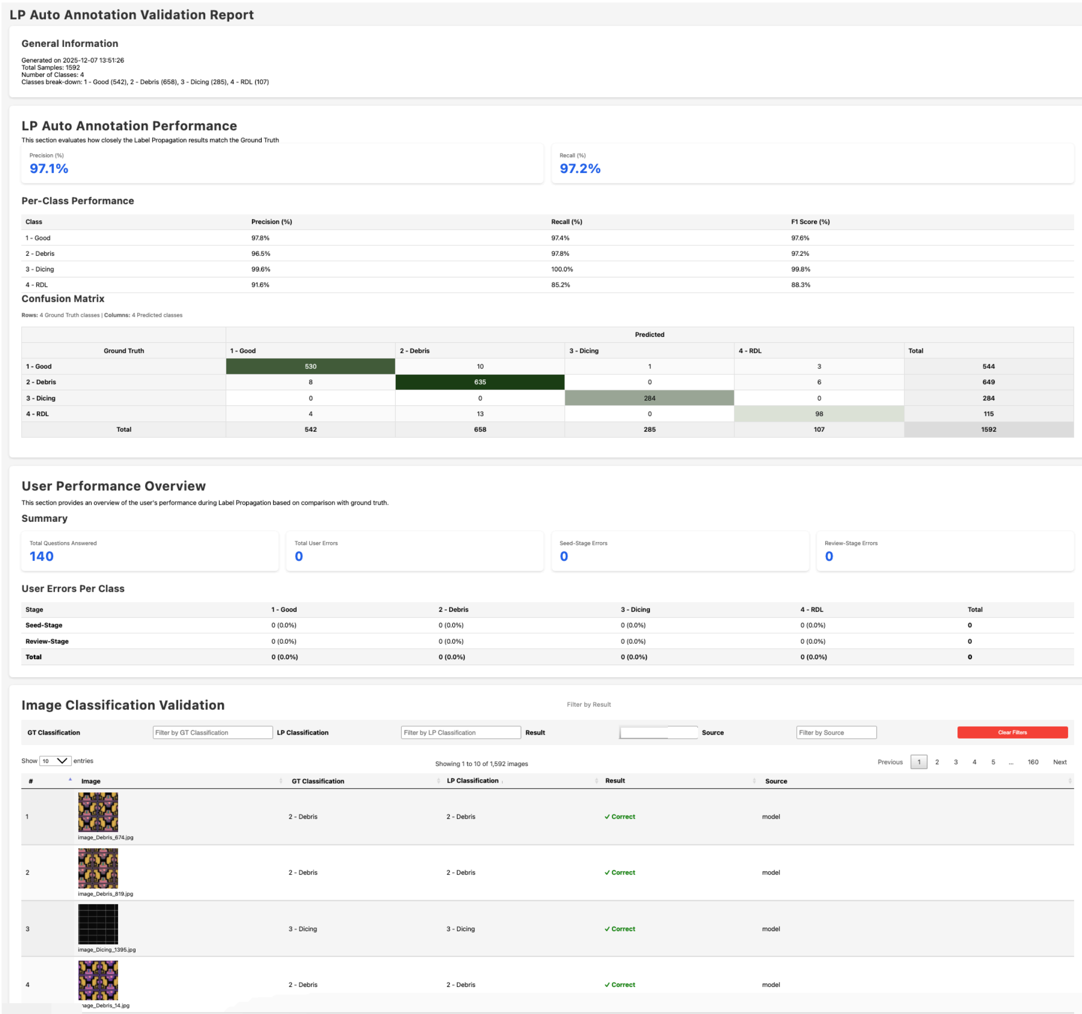
Generate an Evaluation Report

The report includes:

- High-level accuracy metrics and summary statistics
- A breakdown of the user input accuracy
- Confusion matrix showing classification performance across all classes
- Image-by-image comparison of Ground Truth versus Label Propagation results
- Visual examples highlighting agreement and discrepancies

> See Report Structure and Details for more information.

An example evaluation report appears similar to the following:



LP Auto Annotation Validation Report

General Information

Generated on 2025-12-07 13:51:26
Total Samples: 1592
Number of Classes: 4
Classes break-down: 1 - Good (542), 2 - Debris (658), 3 - Dicing (285), 4 - RDL (107)

LP Auto Annotation Performance

This section evaluates how closely the Label Propagation results match the Ground Truth

Precision (%)
97.1%

Recall (%)
97.2%

Per-Class Performance

Class	Precision (%)	Recall (%)	F1 Score (%)
1 - Good	97.8%	97.4%	97.6%
2 - Debris	96.5%	97.8%	97.2%
3 - Dicing	99.6%	100.0%	99.8%
4 - RDL	91.6%	85.2%	88.3%

Confusion Matrix

Rows: 4 Ground Truth classes | **Columns:** 4 Predicted classes

	Predicted				
Ground Truth	1 - Good	2 - Debris	3 - Dicing	4 - RDL	Total
1 - Good	530	10	1	3	544
2 - Debris	8	635	0	6	649
3 - Dicing	0	0	284	0	284
4 - RDL	4	13	0	98	115
Total	542	658	285	107	1592

User Performance Overview

This section provides an overview of the user's performance during Label Propagation based on comparison with ground truth.

Summary

Total Questions Answered
140

Total User Errors
0

Seed-Stage Errors
0

Review-Stage Errors
0

User Errors Per Class

Stage	1 - Good	2 - Debris	3 - Dicing	4 - RDL	Total
Seed-Stage	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	0
Review-Stage	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	0
Total	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	0

Image Classification Validation

Filter by Result

GT Classification Filter by GT Classification LP Classification Filter by LP Classification Result Source Filter by Source Clear Filters

Show 10 entries

Showing 1 to 10 of 1,592 images

Previous 1 2 3 4 5 ... 160 Next

#	Image	GT Classification	LP Classification	Result	Source
1	image_Debris_674.jpg	2 - Debris	2 - Debris	✓ Correct	model
2	image_Debris_819.jpg	2 - Debris	2 - Debris	✓ Correct	model
3	image_Dicing_1395.jpg	3 - Dicing	3 - Dicing	✓ Correct	model
4	ıage_Debris_14.jpg	2 - Debris	2 - Debris	✓ Correct	model

Figure 44: Evaluation Report Example

To create an evaluation report

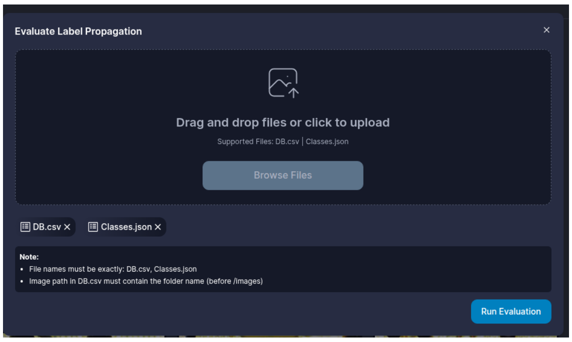
1. Navigate to your dataset in **Visual Layer**.

   Ensure label propagation is complete for the dataset you want to evaluate.

2. Go to the three-dot menu and click **Evaluate Label Propagation**.

> The **Evaluate Label Propagation** option appears only after label propagation has finished running.

The **Evaluate Label Propagation** dialog appears.



3. Click **Browse Folders** and provide the paths to your Ground Truth files:
   - `db.csv` - Your validated image classifications
   - `classes.json` - The defect taxonomy definition

> The Ground Truth files must use the same class names and structure as your Label Propagation configuration. Mismatched taxonomies will prevent report generation.

4. Click **Run Evaluation**.

   A progress notification appears showing the comparison status. The system validates your files, matches image names, compares classifications, and generates the HTML report.

5. When processing completes, a notification confirms the report is ready.

   The report downloads automatically as a ZIP file to your Downloads folder. The ZIP contains:
   - `index.html` - located in the root of the folder , this is the report itself (viewable in Chrome)
   - `js/` - Required JavaScript files (Chart.js and report functionality)
   - `style/` - CSS styling files
   - `DB.csv` - Merged comparison data (including class IDs)
   - `classes.json` - Class mapping reference (maps IDs to class names)
   - `missed_images_data.js` - Data file for mismatched samples

6. Extract the ZIP file.

7. Double click the `index.html` file in the root of the folder to open the report in your Chrome browser.

> Save the extracted report folder for your records. The report remains fully functional offline.

To explore and analyze the report data

- In any table, hover over a cell or row to highlight its position more clearly.
- Slice and dice the **Image Classification Validation** table via various filters.

Report Structure and Details

The evaluation report contains the details and insights as described in this section.

General Information

The report header provides an at-a-glance understanding of how well your label propagation performed across the entire dataset. This summary displays key metrics and metadata about your validation:

Figure 45: General Informaion Example

Field	Description
Generated On	Date and time when the report was created (YYYY-MM-DD HH:MM:SS format)
Total Samples	The total number of sample images that were given for the seed from this dataset
Number of Classes	Total number of classes (labeled) included in the label propagation
Class breakdown	A list of the classes, their class IDs, and the total number of images in each class. For example, 1 - butterfly (92) means the class ID is 1, the class name is butterfly, and 92 images were labeled with this class.

LP Auto Annotation Performance

This section evaluates how closely the Label Propagation results match the Ground Truth, appearing similar to the following example:

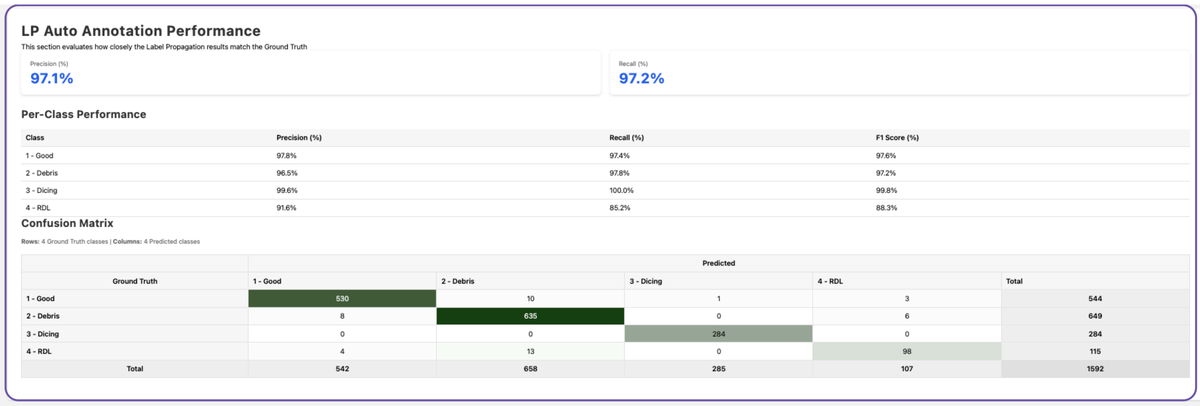


Figure 46: LP Auto Annotation Performance Example

Per-Class Performance

This table displays detailed classification performance metrics for each class.

Metric	Description	Formula
Class	The name of the class for these statistics	
Precision	Percentage of Label Propagation predictions that were correct for this class	True Positives / (True Positives + False Positives)
Recall	Percentage of actual instances that Label Propagation correctly identified	True Positives / (True Positives + False Negatives)
F1-Score	Balanced measure combining Precision and Recall	2 × (Precision × Recall) / (Precision + Recall)

Confusion Matrix

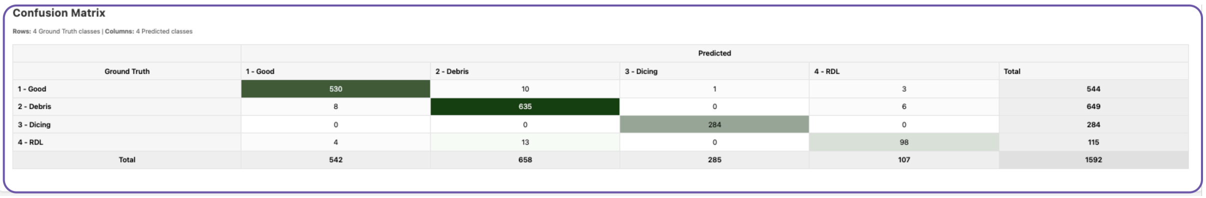
The confusion matrix table displays the predicted classes versus the ground truth. Rows represent the actual Ground Truth labels, while columns show the predicted Label Propagation results.

Each cell in the matrix displays the count of images where a Ground Truth class (row) was predicted as a specific Label Propagation class (column). The diagonal cells (where row equals column) represent correct classifications.

The matrix includes:

- **Row Totals (GT Total)** - The total number of images for each Ground Truth class

- **Column Totals (Predicted Total)** - The total number of images predicted for each class by Label Propagation



Confusion Matrix

Rows: 4 Ground Truth classes | Columns: 4 Predicted classes

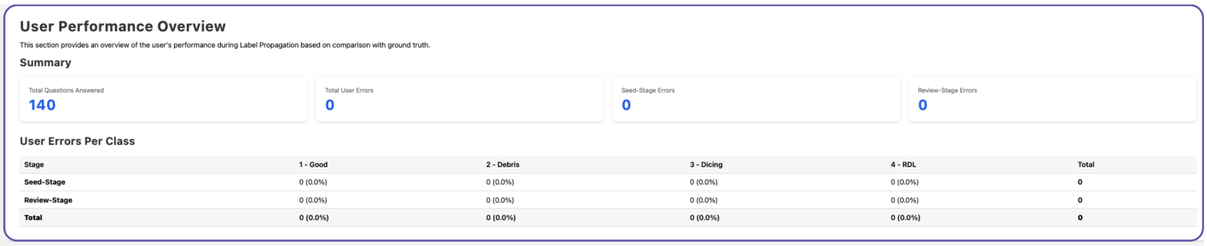
	Predicted				
Ground Truth	1 - Good	2 - Debris	3 - Dicing	4 - RDL	Total
1 - Good	530	10	1	3	544
2 - Debris	8	635	0	6	649
3 - Dicing	0	0	284	0	284
4 - RDL	4	13	0	98	115
Total	542	658	285	107	1592

Figure 47: Confusion Matrix Example

> For datasets with many classes, the matrix displays class IDs with a mapping reference at the top of the table. Use horizontal scrolling to view all columns.

User Performance Overview

This section of the report provides an overview of the user's performance during Label Propagation as compared to the ground truth, which is divided into sub-sections:



User Performance Overview

This section provides an overview of the user's performance during Label Propagation based on comparison with ground truth.

Summary

Total Questions Answered	Total User Errors	Seed-Stage Errors	Review-Stage Errors
140	0	0	0

User Errors Per Class

Stage	1 - Good	2 - Debris	3 - Dicing	4 - RDL	Total
Seed-Stage	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	0
Review-Stage	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	0
Total	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	0

Figure 48: User Performance Overview Example

Summary

The summary itemizes:

- Total Questions Answered - the total number of feedback requests that the user responded to
- Total User Errors - the number of errors that the user made in total
- Seed-Stage Errors - the number of initial seed samples that the user provided incorrectly
- Review-Stage Errors - the total number of feedback requests that the user responded to incorrectly

User Errors per Class

This table displays the details of user errors **per class** for:

- seed stage

- review stage
- in total

Image Classification Validation

The final section displays every image in your dataset with its Ground Truth and Label Propagation classifications side by side. This detailed view enables you to:

- Review individual classification decisions
- Filter images by Ground Truth class, Label Propagation class, match status, or label source
- Identify patterns in misclassifications
- Audit specific defect types requiring attention

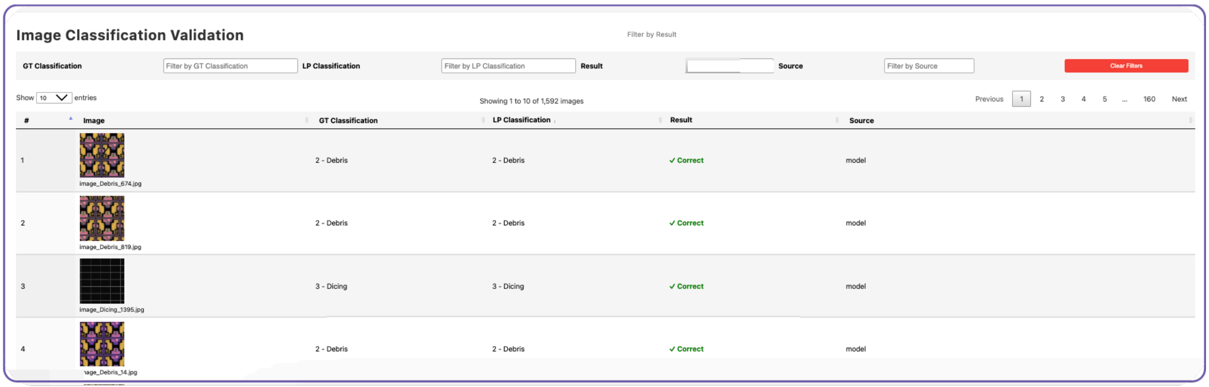


Figure 49: Image Classification Validation Example

To manipulate data in this table:

1. Sort by any column. Click the title of the column to change sorting direction.
2. Filter with these options:
   - GT Classification - view images by a specific ground truth classification
   - LP Classification - view images by a specific label propagation classification
   - Result - filter by whether the label propagation was correct or incorrect
   - Source - filter by the source of the label application - Visual Layer (the model), seed or the user (during review stages)
3. View the continuation of the table from the page arrows at the top right of the table.
4. Click **Clear Filters** to view the entire table.

Export Results

Export detailed labeling results for quality analysis, auditing, or integration with external tools. Export is available at any stage of label propagation, even before reaching train-worthy status. Use this for quality audits or to track annotation progress across iterations.

> This export is optional and not required for model training through the integrated workflow.

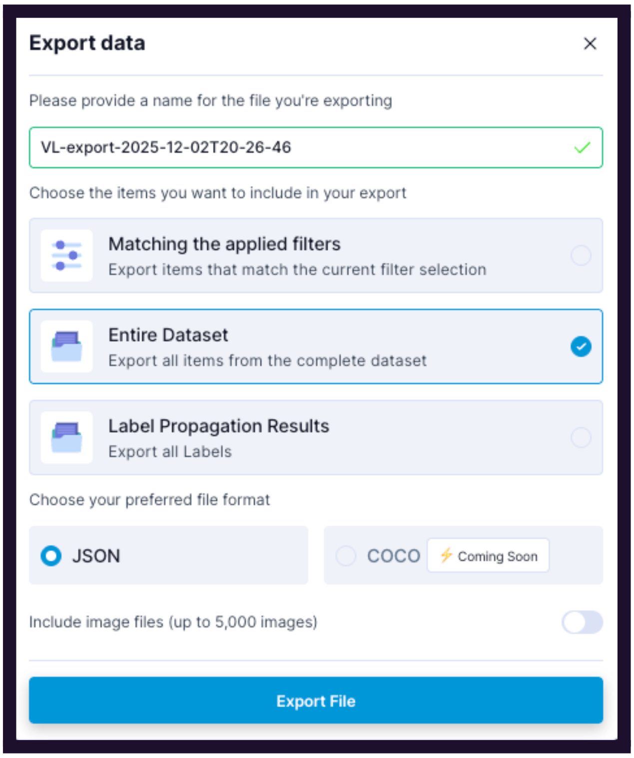
Export Options

Visual Layer generates a JSON file that includes detailed attribution for every labeled image, enabling you to track model accuracy and annotation quality. You can export this file as follows:

Export Option	Description
Export items that match the current filter selection	Export the JSON with only the metadata for a portion of the dataset
Entire Dataset	Export the JSON for the complete dataset
Label Propagation Results	Export the JSON and all label results; this option only appears once you've completed label propagation.
Include image files	Include up to 5,000 images or only export the JSON

To export results

1. Navigate to your dataset.
2. If relevant, filter the dataset to the results that you wish to export.
3. Click from the top right. The **Export data** options open.



4. Configure the export with the relevant options.

5. Click **Export File**.

   The JSON file downloads automatically to your default downloads folder. If you selected to export the images as well, they are downloaded in a ZIP file simultaneously.

Example Structure for Dataset Export

Use this export to work with your dataset, or portion thereof, and all of its metadata.

The structured output appears similar to the following example:

```
{
  "dataset_id": "def456",
  "timestamp": "2025-10-25T10:00:00Z",
  "total_annotations": 950,
  "total_cycles": 3,
  "annotations": [
    {
      "image_path": "images/wafer1/die_101.jpg",
      "label_name": "Surface Defect - Major",
      "source": "VL",
```

```
      "cycle_number": 3
    },
    {
      "image_path": "images/wafer1/die_102.jpg",
      "label_name": "Good Quality",
      "source": "User",
      "cycle_number": 2
    },
    {
      "image_path": "images/wafer2/die_205.jpg",
      "label_name": "Color Inconsistency",
      "source": "Seed",
      "cycle_number": 1
    },
    {
      "image_path": "images/wafer3/die_315.jpg",
      "label_name": "Structural Damage",
      "source": "VL",
      "cycle_number": 3
    }
  ]
}
```

If you export the entire dataset or a filtered portion of the dataset, the JSON file contains:

Field Key	Data Type	Description
info	`Object`	Top-level object containing metadata about the export and dataset.
schema\version	`String`	The version of the export schema.
dataset	`String`	The human-readable name of the dataset.
description	`String`	A brief description of the export operation.
dataset\url	`String`	A URL pointing to the dataset within the **Visual Layer** application.
export\time	`String (ISO 8601)`	Timestamp indicating when the data export was initiated.
dataset\creation\time	`String (ISO 8601)`	Timestamp indicating when the dataset was originally created.

Field Key	Data Type	Description
exported\by	`String`	The user who performed the data export.
total\media\items	`Integer`	The total number of images/media items in the dataset.
media\items	`Array of Objects`	Array containing details for each image in the export (similar to the results array in the standard example, but with richer media details).
media\id	`String`	Unique identifier for the media item (image).
media\type	`String`	The type of media (e.g., “image”).
file\name	`String`	The original file name of the image.
file\path	`String`	The path to the file within the original AOI output structure.
file\size	`String`	The size of the image file.
uniqueness\score	`Number (Float)`	A score indicating how unique the image is compared to the rest of the dataset.
height	`Integer`	The height of the image in pixels.
width	`Integer`	The width of the image in pixels.
url	`String`	A URL to view the specific media item in **Visual Layer**.
cluster\id	`String`	The ID of the visual cluster the image belongs to (from the Exploration features).
metadata\items	`Array of Objects`	Array containing application-specific or custom metadata related to the image.
type	`String`	The type of metadata entry (e.g., “issue”, “image\label”).
properties	`Object`	Object containing specific details for the metadata item type.
issue\type	`String`	Specific classification of the issue (e.g., “mislabels”).
issues\description	`String`	A description or source of the issue.
confidence	`Number (Float)`	A confidence score associated with the metadata/issue.

Example Structure for Label Propagation Export

Use this export to track the source of each label to measure model accuracy and identify areas for improvement:

Use Case	Description
Model acceptance rate	How often VL automatic labels were approved versus corrected
Class-specific performance	Which defect types the model handles well versus those needing more training data
Quality assurance	Full audit trail for compliance and process validation
Training integration	Traceable labeled data for your model pipeline

The structured output appears similar to the following example:

```json
{
  "dataset_id": "abc123",
  "results": [
    {
      "image_id": "img_001",
      "label": "Surface Defect - Minor",
      "source": "VL",
      "confidence": 0.94
    },
    {
      "image_id": "img_002",
      "label": "Good Quality",
      "source": "User",
      "confidence": null
    },
    {
      "image_id": "img_003",
      "label": "Structural Damage",
      "source": "Seed",
      "confidence": null
    }
  ]
}
```

The JSON file contains the following information:

Field	Data Type	Description
dataset\id	`String`	A unique identifier for the dataset.
timestamp	`String (ISO 8601)`	The date and time when the label propagation results were exported.
total\annotations	`Integer`	The total number of images that received a label (either automatic or manual) in this dataset.
total\cycles	`Integer`	The total number of label propagation iterations/cycles completed on this dataset.
annotations	`Array of Objects`	A list containing the labeling result details for each image.
image\path	`String`	The file path of the image within the original directory structure of the dataset.
label\name	`String`	The final class/label assigned to the image.
source	`String`	The origin of the assigned label: user (manual assignment/review), seed (initial user-provided example), or VL (**Visual Layer** automated assignment).
cycle\number	`Integer`	The label propagation iteration number when the label was assigned or last reviewed.

Task Manager

The **Task Manager** provides a unified view of all dataset-related operations across your **Visual Layer** workspace. It tracks processes such as dataset creation, label propagation, and model training, displaying their current status, progress, and any errors that occurred.

Use the **Task Manager** to:

- Monitor active operations in real-time
- Track operation history from the past 14 days
- Identify and troubleshoot failed processes

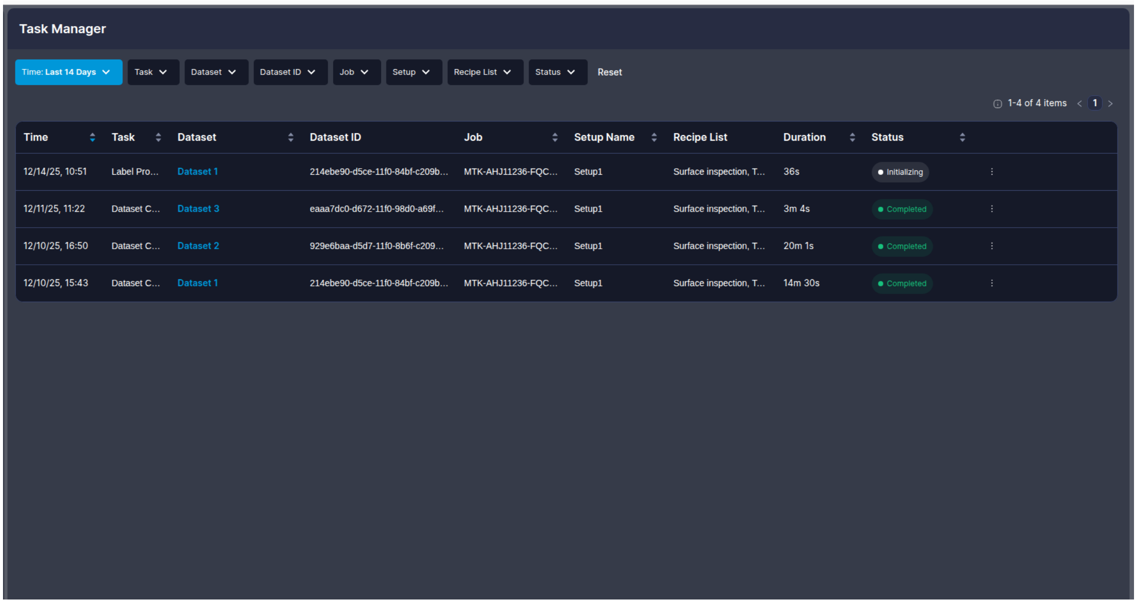
> The **Task Manager** maintains a visible history of all operations performed in the past 14 days.

The **Task Manager** displays near real-time updates on all operations.

To access the **Task Manager**:

1. Go to the **Navigation Panel** on the left side of the screen.
2. Click .

   The **Task Manager** opens, displaying all operations from the past 14 days, similar to the following:



> ⓘ If no operations have been performed in the past 14 days, the table appears empty with a notification indicating no recent activity.

The **Task Manager** displays all operations in a comprehensive table. Each row represents a single task and contains the following information:

Column	Description
Dataset Name	The name of the dataset where the operation is running or was performed. Click the dataset name to navigate directly to its Explore page.
Dataset ID	The unique identifier for the dataset. Use filters to search by specific dataset IDs.
Job	The job identifier from your manufacturing equipment metadata. Each dataset contains a single job value. Use the filter to view tasks by specific jobs.
Setup	The setup identifier from your manufacturing equipment metadata. Each dataset contains a single setup value. Use the filter to view tasks by specific setups.
Recipe	The recipe identifier(s) from your manufacturing equipment metadata. A dataset may contain multiple recipe values. If the recipe name exceeds the cell width, it appears truncated with "..." — hover over the cell to view all recipe values. Use the filter to view tasks by specific recipes.
Task	The type of operation being performed: **Dataset Creation**<!– **Media Addition**, → **Label Propagation**, **Training Model**, or **Re-indexing**. Use the filter to view specific task types.
Start Time	The timestamp when the operation was initiated. Use the filter to view tasks from the last 24 hours, last 7 days, or last 14 days.
Duration	The elapsed time since the operation started (for running tasks) or the total time the operation took (for completed tasks). Sort by this column to identify long-running operations.
Status	The current state of the operation. See the Status Types section below for detailed descriptions of each status. Use the filter to view tasks with specific statuses.
Actions	Retrieve Task ID.

> ⓘ The **Task Manager** tracks operation-level status independently from overall dataset status. A dataset can show **Ready** status in the **Dataset Inventory** even when the **Task Manager** displays completed tasks like **Training** since these operations do not block dataset access.

Status Types

All tasks display one of several statuses to indicate their current state. Each status includes a percentage to show progress through the operation.

See Task Manager Status Types in the **Appendices** for detailed descriptions of all task statuses and their meanings.

Working with the Task Manager Table

The table displays tasks ordered by **Start Time** by default, with the most recent operations appearing first. Each column in the **Task Manager** supports filtering, grouping and sorting to help you quickly locate specific operations:

- **Filtering**: Click the filter icon in any column header, select the desired values, and click **Apply** to update the table.
- **Sorting**: Click the column header to sort tasks in ascending or descending order.

Appendices

User Interface Reference

This reference guide provides detailed descriptions of all **Visual Layer** screens, components, and interface elements. Use this section to understand what each field, button, and area of the platform does.

Home Page: Dataset Inventory

When you log in to **Visual Layer**, the home page displays the **Dataset Inventory**:

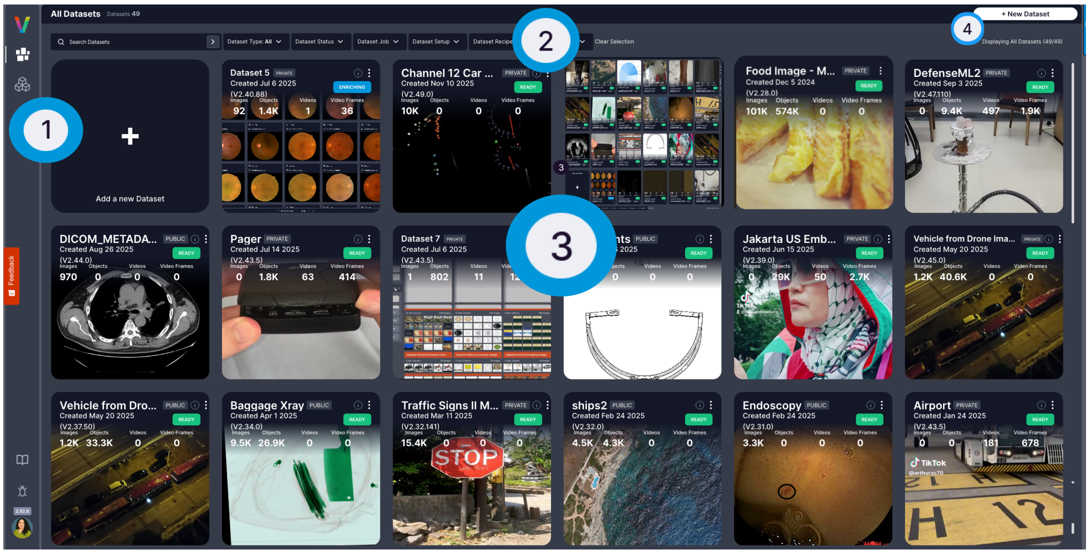


Figure 50: **Visual Layer** Platform

The home page layout consists of:

Area	Name	Description
1	**Navigation Panel**	Navigate to: this area (the **Dataset Inventory**, which is your **Home** page), the **Task Manager**, the **Model Catalog**, and log in/log out
2	**Main Panel**	Search for datasets.
3	**Dataset Inventory**	This is your default Home page, as appears in the example above. The **Dataset Inventory** page displays a complete list of your datasets. Click any dataset to open it.
4	+ New Dataset	Create a new dataset.

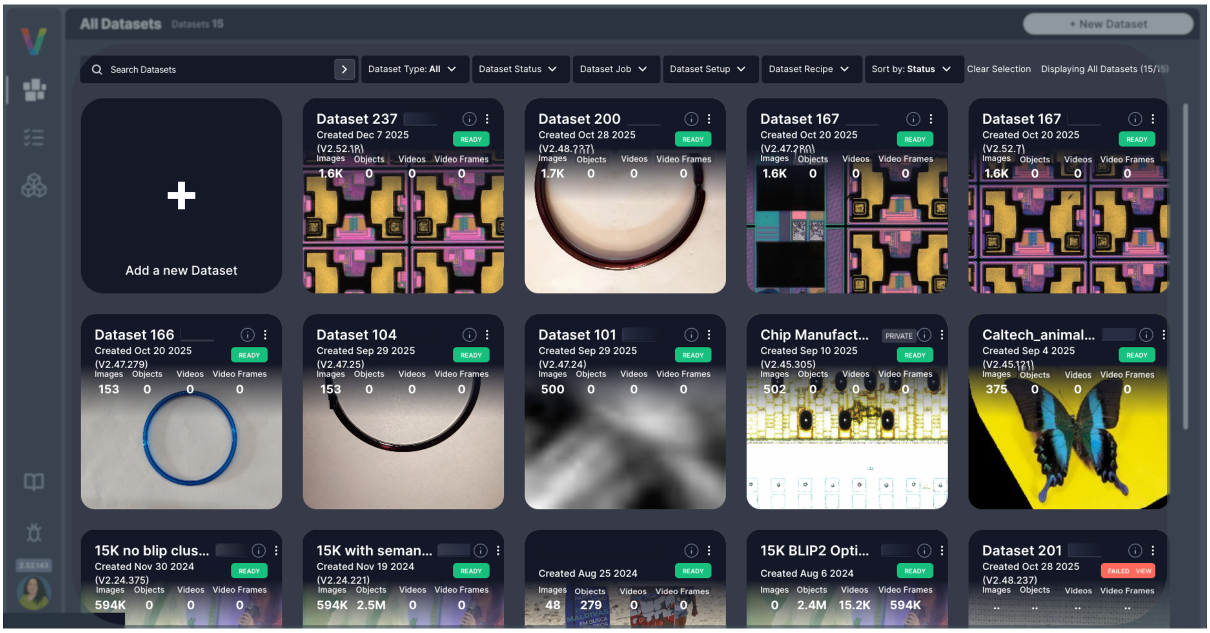
Navigation Panel

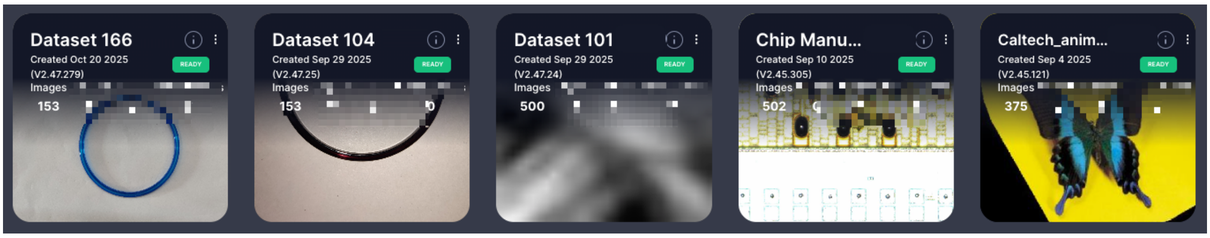
The **Navigation Panel** provides access to all major areas of **Visual Layer**:

Icon	Name	Description
	Dataset Inventory	A complete list of your datasets.
	Task Manager	Monitor and manage all active tasks, jobs, and processing workflows.
	Model Catalog	Browse, deploy, and manage trained models for inference and analysis.
	Your Avatar	Log out of your account.

Dataset Inventory View

The **Dataset Inventory** displays all datasets as cards:





Dataset Card

Each dataset card shows:

Figure 51: Dataset Card Example

Field	Description
Dataset Name	The title of the dataset for easy identification and reference.
Creation Date	The date when the dataset was created and added to your inventory.
Dataset Status	Current state of the dataset: **Ready**, **Processing**, **Failed**, or **Archived**.
Image Count	Total number of images contained in the dataset.
Info Icon ⓘ	Hover to view associated Job Name, Setup Name, and Recipe Names.
Three-Dot Menu	Access dataset actions including **Delete**.

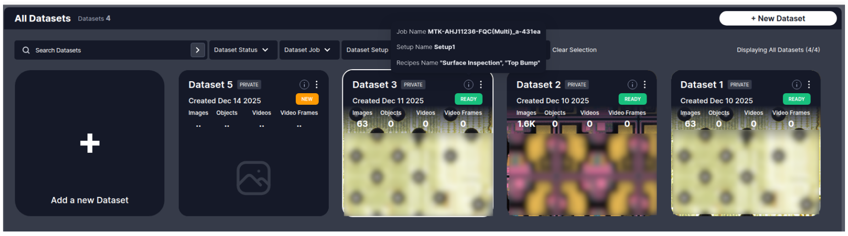
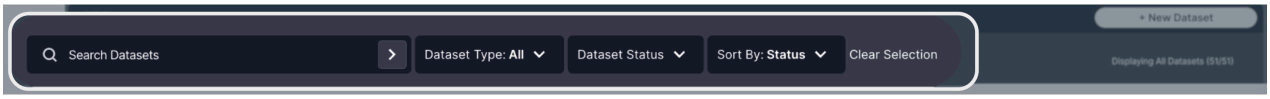


Figure 52: Dataset Metadata Popup Example

Main Panel Controls

The **Main Panel** provides dataset management tools:



Control	Description
Search Bar	Locate datasets by name.
Filter by Job/Recipe/Setup	Filter datasets by production metadata (Job Name, Setup Name, Recipe Name).
Sort By	Arrange datasets based on status, creation date, or other attributes.
Clear Selection	Reset applied filters.

Dataset View

When you open a dataset, the main dataset exploration interface loads:

The dataset view consists of:

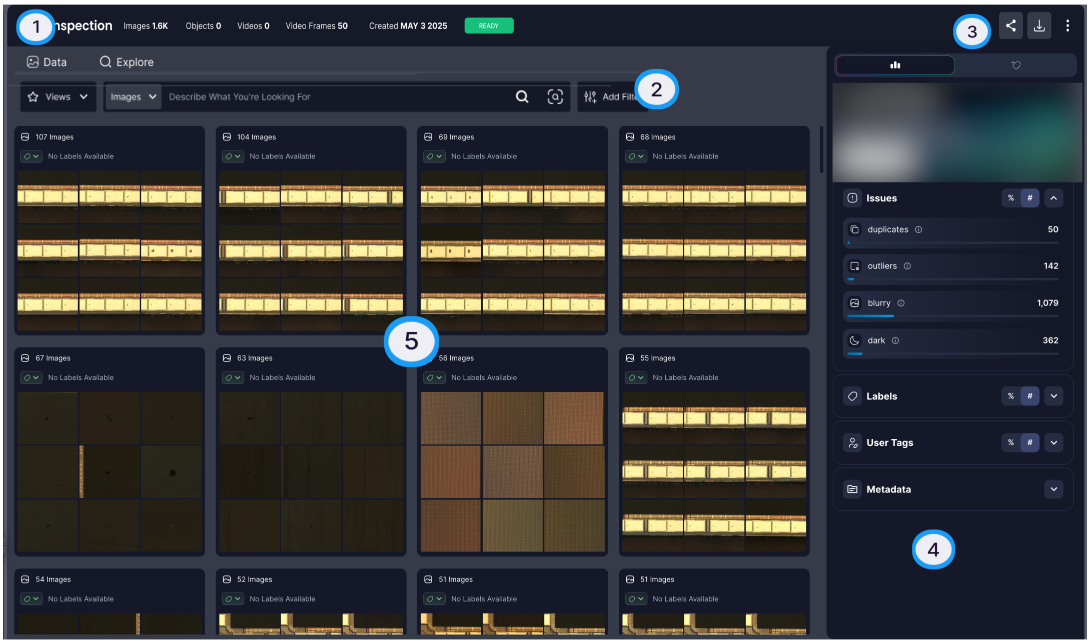


Figure 53: Dataset View with Details

Area	Component	Description
1	**Tabs**	Navigate between Explore, Data and Views.
2	**Filter Panel**	Apply and combine search criteria.
3	**Action Bar**	Access operations like exporting.
4	**Details Sidebar**	View metadata and insights, and access label propagation.
5	**Thumbnail Grid**	Visual representation of your data in clusters.

Dataset Metadata

Hover over the info icon next to the dataset name to view production metadata:

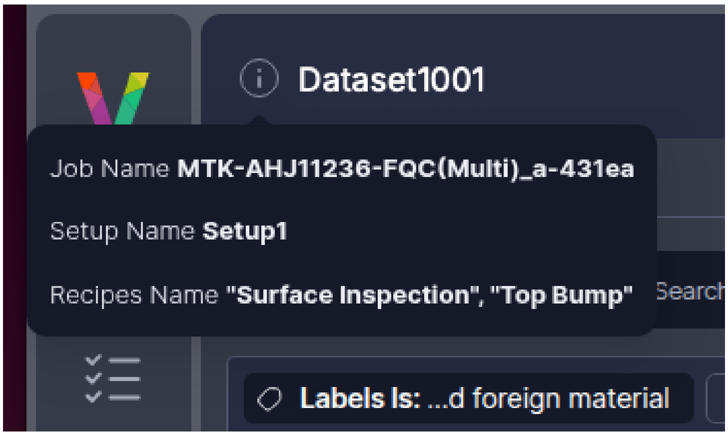


Figure 54: Dataset Metadata Popup

Field	Description
Job Name	The production job identifier from the folder metadata.
Setup Name	The setup configuration identifier from the folder metadata.
Recipe Names	The recipe name(s) used during inspection.

Cluster View

Visual Layer automatically groups similar images into clusters during dataset creation. Each cluster is displayed as a preview card:

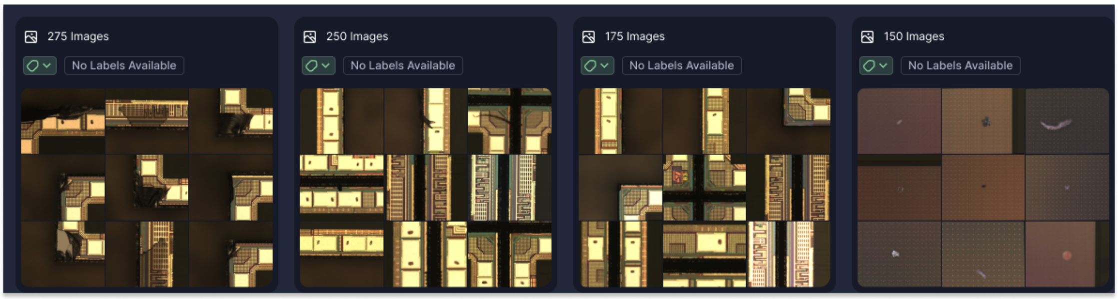


Figure 55: Cluster preview cards

Cluster Card Element	Description
Number of items	The total number of items that match your current filters
Prominent labels	The most frequent labels within the cluster

Cluster Card Element	Description
Representing images	Thumbnails of a few key example images that visually summarize the cluster
Visual Similarity Icon	Hover over a cluster to access visual similarity search

Filter Menu

Access filters by clicking in the dataset toolbar:

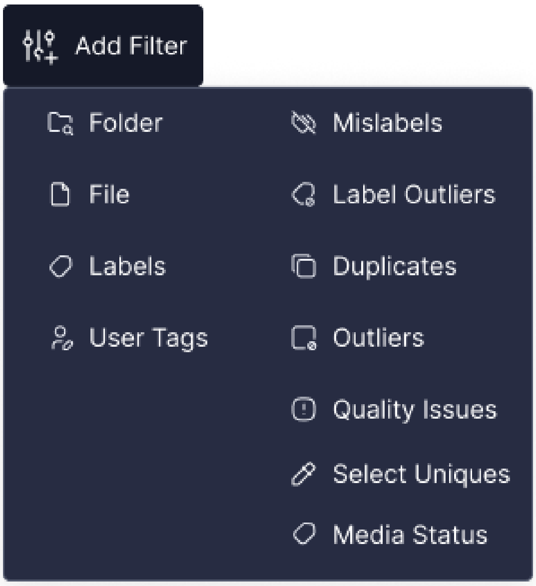


Figure 56: All Filters

Available Filters

Filter Category	Description	Configuration
Folders	Filter by specific AOI output folders within your dataset	Check the folders to include
Files	Filter by specific image files or file patterns	Enter filename or pattern to match
Labels	Filter by assigned label classes	Select which classes to include or exclude
User Tags	Filter by custom tags you've applied to images	Select tags from your tag list
Duplicates	Show only duplicate or near-duplicate images	Toggle on to show duplicates only

Filter Category	Description	Configuration
Outliers	Show images that differ significantly from typical patterns	Toggle on to show outliers only
Quality Issues	Filter images flagged with quality problems	Select issue types (blur, overexposure, underexposure, etc.)
Select Uniques	Show only unique images (exclude duplicates)	Toggle on to show unique images only
Insertion Time	Filter by when images were added to the dataset	Set date ranges for upload time
Media Status	Filter by processing or indexing status	Select status types (pending, processing, ready, error)

Applied Filters Display

Applied filters appear in the **Query Panel** above the image grid:

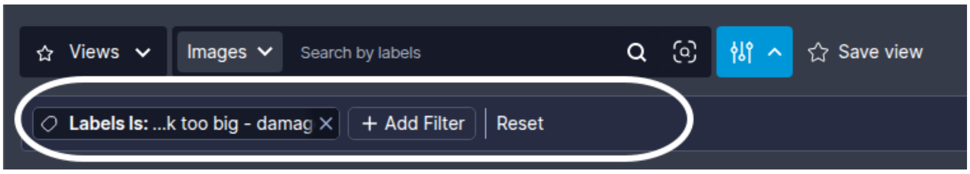


Figure 57: Applied Filters

Control	Description
Filter Chip	Each applied filter shows as a chip with its criteria
X Button	Click to remove an individual filter
Clear All	Remove all applied filters at once

Multiple filters combine using **AND** logic.

Visual Similarity Search

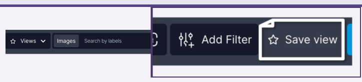
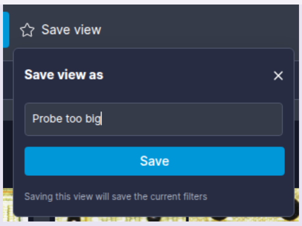
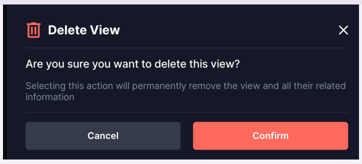
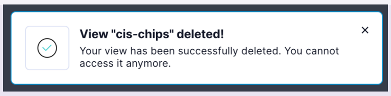
Visual Layer enables similarity search using deep visual embeddings:

Search Method	Action
From cluster or image in cluster view	Hover over a cluster or the image and click at the bottom of the card.
From the Image Details Page	Click a single image to open its details page, then click Find Similar from the **Action Bar**.

Search Method	Action
Using an External Image	Click  near the search bar inside your dataset, upload an image from your device, optionally crop it, and start the search.

Views Tab

The **Views** tab displays saved filter combinations and search queries:

Control	Description
Save as View Button	 Saves current filters and search criteria as a reusable view
View Name Dialog	 Enter a descriptive, unique name for the view
View Card	Click any saved view to load its filter combination
Three-Dot Menu	Access view actions including **Delete**
Delete Confirmation	
Delete Notification	

Views are immutable. To modify a view, delete it and create a new one with updated criteria.

Dataset Creation Stages

During dataset creation, **Visual Layer** prepares the dataset for exploration through the following stages:

Status	Description
NEW	Dataset initialization started
UPLOADING	Upload request acknowledged, data transfer in progress
INITIALIZING	Pre-processing setup in progress
PRE_PROCESSING	Data prepared for indexing and analysis
INDEXING	Visual indexing and AI-based analysis in progress: learning distribution patterns and creating enhanced embeddings tailored to your data for label propagation accuracy
SAVING	Final data save to system in progress
READY	Dataset complete and available for exploration
FATAL_ERROR	Irrecoverable failure during dataset creation; check error details in notification or **Task Manager**

Dataset Operation Statuses

After dataset creation (or dataset update), dataset status appears on its card in the **Dataset Inventory** and inside the dataset at the top right. Statuses indicate the current state and availability of dataset operations:

Status	Description
READY	No blocking operations are running. All actions are available.
UPDATING	A blocking operation is running (**Label Propagation** or **Re-indexing**). Most dataset actions are unavailable until the operation completes.
CREATING	The dataset is being created. No dataset actions are available until creation completes.
RESUME	User action is required to complete dataset creation. Data upload started but **Create Dataset** was not clicked.
PENDING	New media was added and requires re-indexing.

> A dataset can show **Ready** status in the **Dataset Inventory** even when the **Task Manager** displays completed tasks like **Training** since these operations do not block dataset access.

Task Manager Statuses

Tasks are operations you perform in the system, such as creating a dataset, updating an existing dataset or running label propagation. All tasks in the **Task Manager** display one of the following statuses to indicate their current state. Each status includes a percentage to show progress through the operation:

Status	Description
Initializing	The operation is performing initial validations, uploading data, or downloading resources. Progress percentage reflects completion of initialization steps.
Running	The operation is actively processing. Progress percentage reflects completion of the overall operation.
Succeeded	The operation completed successfully. Progress shows 100%.
Failed	The operation encountered an error and could not complete. Hover over the status to view the error message explaining what went wrong.
Aborted	The operation was manually stopped by a user before completion.
Waiting for Review	The label propagation operation completed and is ready for your review. Navigate to the dataset to begin reviewing propagated labels. This status applies only to **Label Propagation** tasks.

Model Training Statuses

When you initiate model training, **Visual Layer** displays real-time progress updates showing the current status and completion percentage (0-100%). The training process progresses through the following stages:

Status	Description
Waiting for Start	Training job is queued and waiting to begin.
Running	Model training is actively in progress. Progress updates in real-time.
Completed Successfully	Training finished successfully. Model is ready in the Model Catalog.
Exited	Training process exited before completion.
Failed	Training encountered an error. Hover over the status to view error details.

Progress Indicators

Visual Layer provides multiple progress indicators throughout the label propagation workflow to help you track completion status and make informed decisions about when to finish. These indicators appear during both seed creation and iterative review phases.

Seed Creation Progress

During seed creation, visual indicators show when you've met the minimum requirement of 5 examples per class. Each class displays a progress indicator that fills as you add examples:

Figure 58: Seed Selection Progress Indicator

When all classes show complete indicators (5+ examples each), the **Submit** button becomes enabled, allowing you to begin label propagation.

Label Propagation Progress

Visual Layer requires 100 labeled images per class as the minimum threshold for training readiness. The platform provides three distinct progress indicators to track your advancement:

Train-worthy Progress

Shows progress toward the 100 images per class training readiness goal. When this reaches 100%, you have met the minimum threshold required to train models downstream.

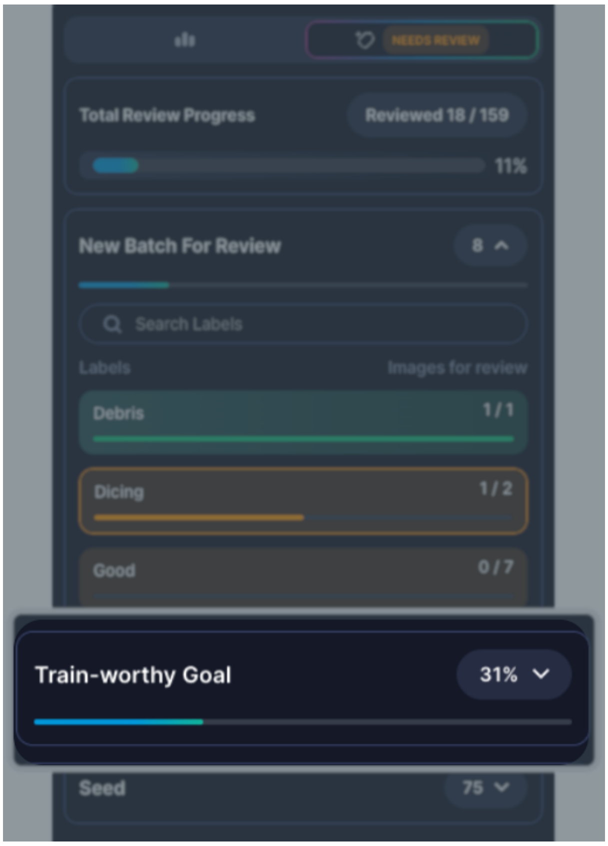


Figure 59: Train-worthy Progress

This indicator focuses on whether you've reached the training threshold, not on labeling your entire dataset.

Overall Progress

Shows the percentage of your entire dataset that has been labeled, regardless of the per-class training threshold. This indicates how much of your total dataset has received labels.

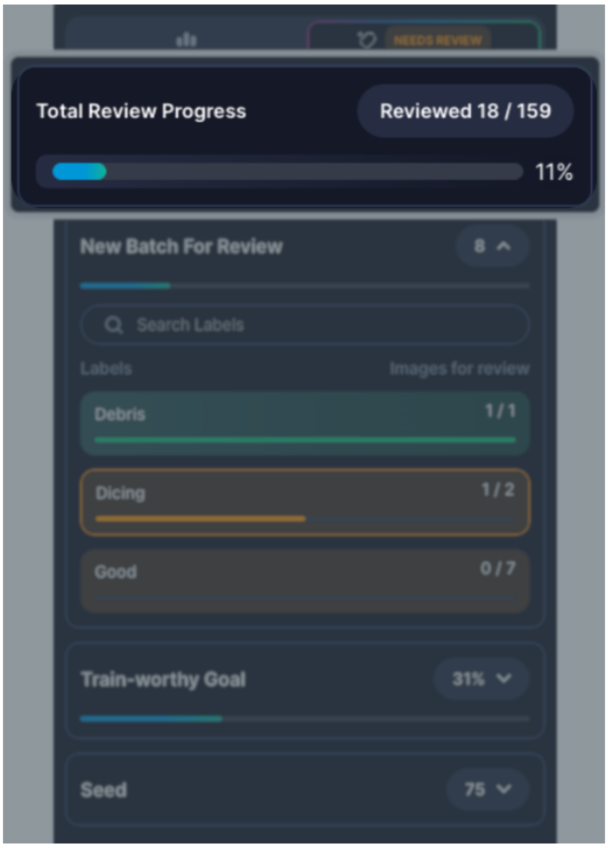


Figure 60: Overall Progress

Use this indicator to understand dataset completion as a whole, independent of training requirements.

Per-Class Progress

Shows how many labeled images each individual class has toward the 100-image minimum threshold. Use this to identify which classes need more labeled examples before you can train models.

Figure 61: Per-Class Progress

This granular view helps you prioritize review efforts for classes that haven't reached the training threshold yet.

Troubleshooting Common Issues

This section provides solutions to errors and problems you might encounter while working with **Visual Layer**. Each issue includes an explanation of what went wrong and specific steps to resolve it.

Dataset Creation and Media Upload Errors

Camtek automated optical inspection (AOI) scans produce a standardized directory containing inspection results, production metadata, class/label definitions, and the associated images organized into subfolders. **Visual Layer** processes the output from Camtek AOI machines to rapidly train defect detection models.

When uploading folders or adding media to datasets, **Visual Layer** validates that all required files exist and are structured as expected. If validation fails, you'll see one of these error messages:

Error	What It Means	How to Fix
Missing required files	Your folder is missing one or more required files: db.csv, metadata.json, classes.json, or db_export_*.json	Check that the AOI scan completed successfully. All four file types must be present in the folder.
Invalid file naming	The db_export file doesn't follow the expected naming pattern	Rename the file to match the pattern: db_export_[id].json (for example, db_export_12345.json)
JSON parsing error	The classes.json or db_export file contains syntax errors	Open the file in a JSON validator to identify and fix formatting issues (missing commas, quotes, brackets, etc.)
Cross-validation failure	Job Name, Setup Name, or Recipe Name don't match between files within the same folder	The metadata.json file may be corrupted. Try regenerating the scan from the AOI machine.

Error	What It Means	How to Fix
Job Recipe mismatch	Job Name, Setup Name, Recipe Name, or Tool List differs between folders you're trying to combine—or doesn't match the original dataset when adding media	For new datasets: Only combine folders from the same production run. For adding media: Ensure the new folder came from the exact same Job/Setup/Recipe as the original dataset.
ImageName mismatch	Filenames listed in db.csv don't match the actual image files in the folder	Check that the ImageName column in db.csv contains exact filenames with extensions. Fix any typos or missing extensions.
Class taxonomy mismatch	The classes.json file defines different classes than the original dataset	When adding media to an existing dataset, you must use the identical classes.json file from the original dataset creation. Classes cannot be changed when adding media.

Label Propagation

These issues might arise during seed creation and iterative review. Most problems stem from insufficient or unrepresentative seed examples.

Problem	Description	Resolution
Inaccurate results for certain defect types	The model consistently mislabels specific defect types. High-confidence automatic labels are frequently incorrect during review.	Add more diverse seed examples showing the full range of that defect type's visual variations. Avoid using edge cases or ambiguous examples as initial seeds. Reset if needed to start with improved seeds.
Cannot drag multiple selected items to assign labels	You're trying to drag multiple selected images to a class label, but only single items can be dragged.	Use the Select & Add workflow instead: select multiple images using checkboxes, click **Assign Label**, then choose the target class from the dropdown.

Frequently Asked Questions

Q: Why do review batches keep appearing? How many times do I need to review?

A: Label propagation is an iterative process where each review round improves the model's accuracy. Continue reviewing until Visual Layer reaches high confidence across your dataset or you're satisfied with the labeling progress. The number of iterations varies based on dataset complexity and seed quality.

Q: Why can't I edit my seed examples after starting label propagation?

A: Seeds become immutable once propagation begins to ensure consistency throughout the iterative learning process. If you need to change seeds or classes, use Reset to start over. Note that this permanently deletes all label propagation progress but preserves your original dataset.

Q: What do the source indicators (VL, User badges) mean?

A: These badges show the origin of each label. **VL** means the label was assigned by Visual Layer with high confidence. **User** means you reviewed and confirmed the label in a previous iteration. Seed examples appear without a badge. See Reviewing and Providing Feedback for complete definitions.

Q: Can I run label propagation on the same dataset multiple times?

A: Yes. However, each new run will overwrite the previous results within that dataset.

Q: What happens to images I marked as "Ignore"?

A: They are excluded from label propagation, placed in an "Unlabeled" cluster, and don't count toward training readiness or review quotas.

Q: Can I export data before reaching "Train Worthy" status?

A: Yes. You can export label propagation results at any point using Export, even if you haven't reached the 100 images per class threshold.

Q: How do I know if my seed examples are good quality?

A: Monitor first-iteration results. If many high-confidence automatic labels are incorrect during your first review, your seed likely needs improvement. Reset and select clearer, more representative examples.

Q: Can I combine datasets from different Jobs/Setups/Recipes?

A: No. Each dataset must contain folders with identical Job, Setup, and Recipe metadata. Create separate datasets for different production runs, then filter and organize them using the metadata filters.

Q: Why are some folders grayed out when I try to add media or create a dataset?

A: Visual Layer automatically validates metadata compatibility. Grayed-out folders indicate their Job, Setup, Recipe, or Tool List metadata doesn't match the other selected folders. Hover over the info icon to view the specific mismatch. You can either select only folders with matching metadata, or create separate datasets for incompatible production runs.

Q: How do I fix incorrect seed examples or classes after starting label propagation?

A: Use Reset to start over with new seeds or classes. This permanently deletes all label propagation progress but preserves your original dataset. Seeds become immutable once propagation begins to ensure consistency throughout the iterative learning process, so resetting is the only way to change them.

Q: Why haven't I reached train-worthy status after multiple review rounds?

A: Visual Layer requires a minimum of 100 images per class for training readiness. Continue review rounds until this threshold is met. If progress is very slow, verify you have sufficient unlabeled data in your dataset and consider adding more images. The number of iterations needed varies based on dataset size, complexity, and seed quality.

Terms and Definitions

Term	Definition	Example/Context
Class	Each distinct type of label in your classification system. Classes are the buckets that organize your labels. You need at least two classes for any classification task, and each class needs enough examples to teach the system what belongs in that category.	If classifying defects, your classes might be “Scratch,” “Contamination,” “Void,” and “Pass”
Class Taxonomy	The complete hierarchical structure of all possible classes in your classification system, defined in classes.json. This cannot be changed after dataset creation.	Your defect taxonomy might include top-level classes like “Physical Defects” and “Contamination,” each with specific sub-types
Confidence Score	A numerical measure (0-100%) indicating how certain the system is about an automatically assigned label. Higher scores mean greater certainty.	An image labeled “Scratch” with 95% confidence versus one with 60% confidence
Dataset	A collection of images and their associated metadata organized for analysis and labeling. A dataset can contain thousands of images from one or more AOI output directories.	All inspection images from a production run of Product X across multiple scans
Embedding	The technology that enables Visual Layer to find visually similar images and automatically label thousands from just a few examples, dramatically reducing manual labeling work.	Visual Layer analyzes your images to understand visual patterns, allowing it to recognize that Image A looks more like your “Scratch” seeds than your “Void” seeds

Term	Definition	Example/Context
Ground Truth	Verified, accurate labels that serve as the authoritative reference for training and evaluating machine learning models. Ground truth is established through expert review and validation, providing the foundation for measuring model accuracy and performance. In Visual Layer, your seed examples and reviewed labels form the ground truth for label propagation.	A set of 100 manually verified defect images where domain experts have confirmed each label, used to measure how accurately your model classifies new images
Iteration	A complete cycle of seed selection, automatic labeling, review, and refinement. Running multiple iterations improves labeling accuracy over time.	First iteration: provide seeds and review results. Second iteration: adjust seeds based on results and review again
Job/Production Metadata	The production configuration parameters that define how inspection was performed. All folders in a dataset must share the same Job, Setup, and Recipe to ensure consistent scanning conditions.	Job: “Product-X-2025”, Setup: “Standard-Inspection”, Recipe: “High-Resolution-Scan”
Label	The category or classification assigned to a single image. Think of it as the answer to “what type is this?” for one specific data point. A label is an instance of a class.	This particular image is labeled “Scratch” (the label is the specific assignment to this one image)
Label Propagation	A semi-supervised learning process that combines automated pattern recognition with human oversight to label large datasets efficiently. The system learns from your seed examples, automatically assigns labels based on confidence scores, and generates review batches of uncertain images for your verification. Each iteration refines the model as you approve or correct labels, creating a feedback loop that improves accuracy until the system can label independently.	After providing 10 seed examples of “Void” defects, the system automatically labels 800 similar images with high confidence, sends 100 uncertain images for your review, then uses your feedback to relabel and improve accuracy in the next iteration

Term	**Definition**	**Example/Context**
Metadata	Descriptive information about your images and inspection process, stored in db.csv and db_export_*.json files. Includes production parameters, timestamps, and file locations.	Information like scan date, machine ID, product type, and image file paths
Seed	The initial labeled examples you provide for each class. These training examples show the system what each class looks like in practice. Seeds are the foundation that makes automatic labeling possible. Without good seed examples, the system won't know what patterns to look for.	Selecting 5-10 clear images of scratches to teach the system what "Scratch" defects look like